

Labbook Week 1

30 September 2025 09:15

First "project day"

- Planning to have project work days, where we work together in library, every Tuesday

Admin and Planning

- Deciding to start with an outline plan of what we aim to do in on a rough week by week basis
 - o Good to present to supervisor for first meeting

Main Steps

Software setup

- Learning and setting up local builds of docker
- Build local images of openMC environments
- [Optional: Setup a git repository for good source control \(+ education \)](#)

Literature Review

- Review general fusion motivations for breeding blankets, state of technology and main players / stakeholders, main types for breeding blankets
 - o ITER (big)
 - o China (check EAST etc.)
 - o US (Commonwealth Fusion, Liquid Lithium)
 - o UK (Oxford / Harwell Campus, Oxford Sigma (liquid lithium))
- Review of tritium breeding literature (critical)
- Brief review of Software, MC, openMC developed by STFC - same amount of detail as experimental methodology, instead you discuss how MC code works.

Prototype Model Phase

- Work towards basic "complete loop" model
- Simple geometry, typical materials, focus on building reliable models
- Modularise code and distribute packaged functions for these basic models (with function inputs ready for adjustments later on)

Prototype Evaluation Phase

- Data analysis (TBR is the main goal)
- Critical evaluation of results (attempts to check if physical)
- Plan next research areas given the results
- Plan areas of the model to iterate and develops

Research Modelling Phase

- Depends on what we have time for
- Parameter sweeps
 - o Sweeps of breeder blanket thickness (aided by good modularisation of code)
 - o Sweep enrichment of different materials (Lithium)
 - o Material Ratios (amount of lead)
- Research into different materials (research on custom material properties)
- [Extension: Adjusting geometries](#)

Research Evaluation Phase

- Data analysis (TBR is the main goal)
- Critical evaluation of results (attempts to check if physical)
- Presentation of results for our simulations against different parameters / materials
- Conclusions on findings for improving TBR ratios
- [Optional: Viability assessments of certain paths](#)
- Software outcomes and discussion of possible extensions

Extension: Further Software Developments

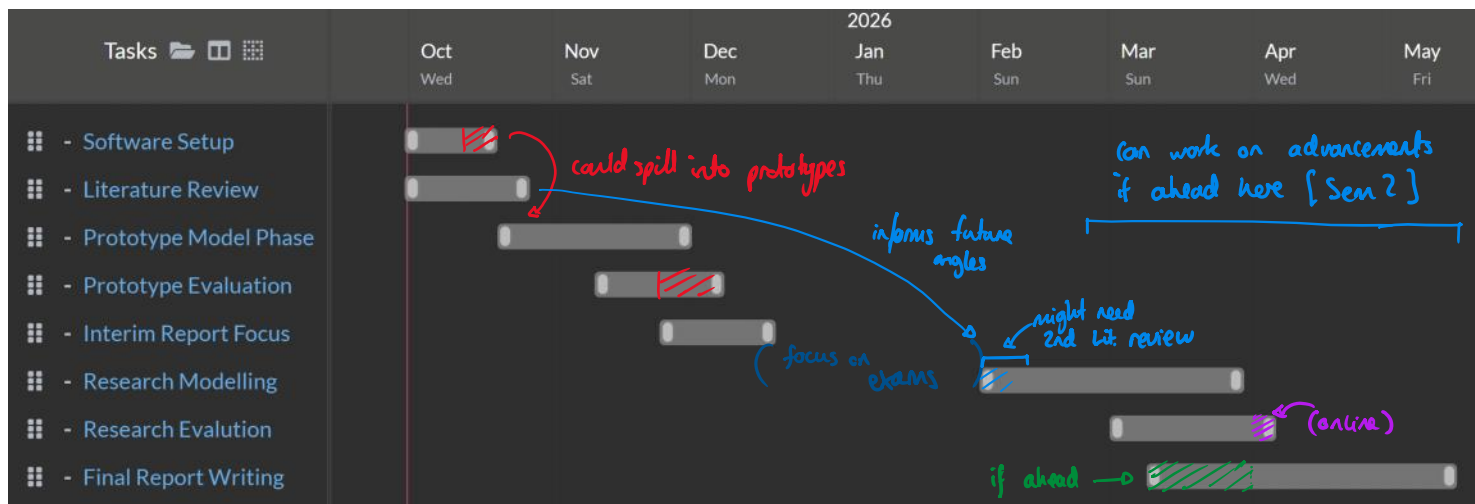
- Only if time permits
- Attempting training of surrogate model based on research data
- Attempting RL model to automate and iterate optimum TBR given certain inputs.

Report Writing

- Abstract and introduction
- Theory Section before literature review as broad intro to fusion
 - o Keep it to semester 1 foundations
- Literature Review
- Implementation / Process (software explanation) / Data Collection
 - o Github reference is generally sufficient
- Results
- Analysis / conclusion

How to we split work? If we both do a computational project? Can we just work on the same main phases, but just split the actual detailed tasks

- Eg: I start trying to sweep materials, you start trying to sweep blanket thickness



Beginning Literature Review

- Evaluating Paper that covers a very similar project using the same software: arXiv:2310.00220v1 [physics.plasm-ph] 30 Sep 2023

Outcomes / learnings from reading that paper + triggered follow up research:

- List of common tested blanket materials:

Solid Breeder Blankets

These are also referred to as ceramic breeders in the paper.

- Li₄SiO₄ (Lithium Orthosilicate)
- Li₂O (Lithium Oxide)
- Li₂ZrO₃ (Lithium Zirconate)
- Li₂TiO₃ (Lithium Titanate)

Liquid Breeder Blankets

- Li₁₇Pb₈₃
- PbLi₁₀₀
- Pure Lithium

Molten Salt Breeder Blankets

- FLiBe (a mixture of LiF and BeF₂)

- **Best Breeder:** The liquid breeder material Li₁₇Pb₈₃ was found to be one of the best options, as it provides a very high Tritium Breeding Ratio (TBR), is not toxic like FLiBe, and contains a low concentration of lithium, which is a scarce material.
- **All about Li-6:** The concentration of Lithium-6 is the most critical factor influencing the TBR. However, the relationship is logarithmic, so increasing enrichment beyond 40% offers diminishing returns, only minimally increasing the TBR.
 - Most critical factor due to cross sections ?
- **All about D-T and Lead:** D-T (Deuterium-Tritium) fusion is significantly more effective for tritium breeding than D-D (Deuterium-Deuterium) fusion. This is because the high-energy neutrons from D-T reactions (14.1 MeV) are much better at causing neutron multiplication in materials like lead compared to the low-energy neutrons from D-D reactions (2.5 MeV).
- **Consider Dose Rate, thermal and mechanical properties too:**
 - While lead-lithium (PbLi) breeders yield the highest TBRs, the molten salt blanket FLiBe is superior in terms of efficiency and dose rate, absorbing radiation much more effectively and meeting recycling limits faster.
 - Ceramic breeders, such as Lithium Orthosilicate (Li₄SiO₄), offer TBRs close to 1 and possess good thermal and mechanical properties that make them resistant to the extreme conditions inside a tokamak.

- Details on paramak modules to use

Criticisms:

- Need more cross section graphs
 - Li₆ compared to Li₇ (need to verify Q values + cross reference nuclear physics)
 - Cross sections related to dose rates
- Non comprehensive data on simulation results
 - Can include more data on enrichment sweeps
 - Poor data on neutron multiplier concentration, confusing figure chosen
- No sweeps of thicknesses of blanket / geometry

Actions:

- Further reading on MC methods and implementation
- Reading into paramak modules
- Cross section data to obtain + reading into dose rate research

Decisions:

- **Only consider DT plasma + Lead**, others just seem <<1
- Focus research on having a more comprehensive set of sweep results
 - Especially a more detailed investigation into multiplier concentration
- Thicknesses is an untapped powerful field to try and focus on.

Link to Drive of Shared Documents:

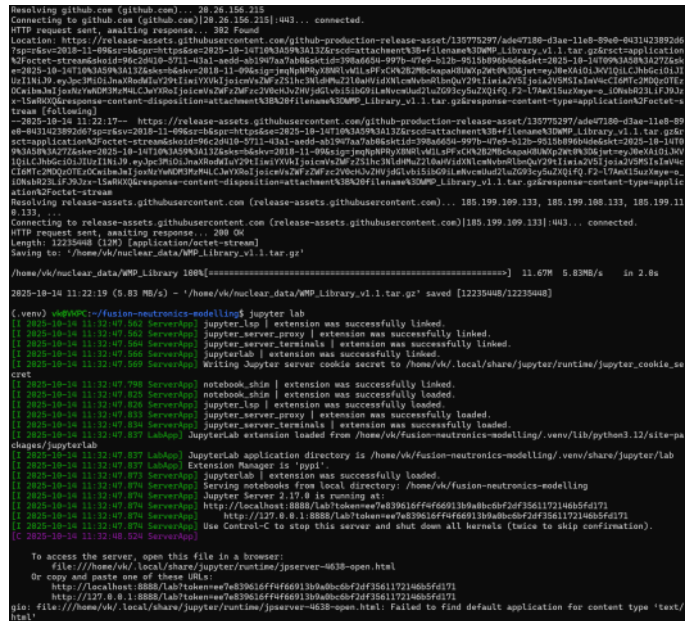
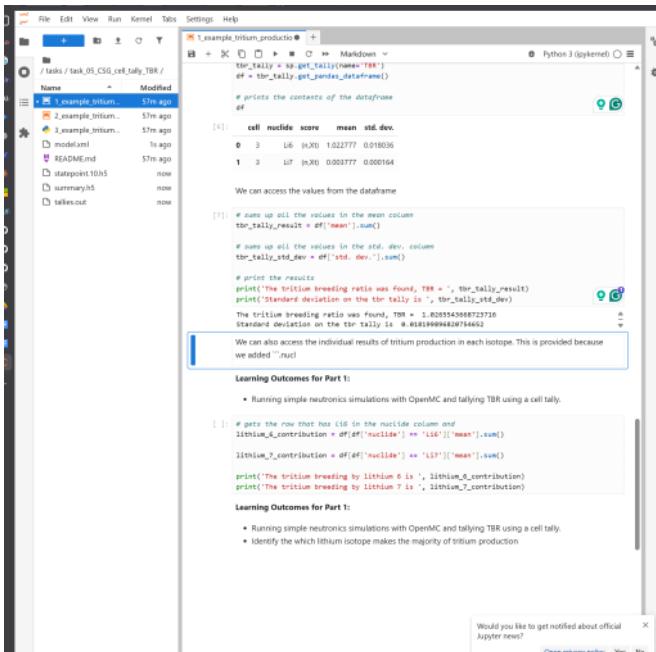
<https://drive.google.com/drive/u/0/folders/19LqvtoSB7r2HaBN0A3UFINr3q7Z7n65p>

- Learning how to interface via python via command line (pip install, check version etc.)
- Github
- VS Code
 - o VS Code Python Setup
- Lookup Virtual Environments
 - o Setup a Virtual Environment via PDM?
- Attempt Pushing the PDM project to Github so I can access.

The primary objectives for this session were twofold:

- This session began with a deep dive into the established body of research surrounding fusion reactor breeder blankets. My lab partner focused on broad literature discovery using Zotero and the university's EndNote subscription, while my review was targeted at the technical specifics of prior simulation work. Key areas of investigation included:

- [illegible]



4.0 Results & Key Milestones

- A comprehensive literature review has informed our project's technical direction and confirmed our choice of simulation software.
- A working, local OpenMC simulation environment was successfully configured and validated on my primary machine using WSL2.
- A clear plan for the project's software stack (WSL2, Python, VS Code, Git) and referencing tools (Zotero) has been established and communicated with my lab partner.

5.0 Next Steps

The immediate next step is to streamline the development workflow. The current WSL environment, while functional, is accessed solely through the Ubuntu terminal. The plan for the next session is to fully integrate this environment with VS Code, establish our own version-controlled project repository on GitHub, and document the setup process for reproducibility.

Supervisor Advice for Paper Parsing Software

Reference Manager:

- ☒ Zotero
- ☒ End Note (Library Provided)

Text Editor Suggestions:

- ☒ Easier: VS Code Extension
 - Intermediate: Quarto (local install / vs code extension)
 - Advanced: Techlive (+ Natlib)

Google Scholar account:

- ☐ Can keep track of references

Link to Drive of Shared Documents:

<https://drive.google.com/drive/u/0/folders/19LqvtosB7r2HaBN0A3UFINr3q7Z7n65p>

Labbook Week 3

14 October 2025 11:48

1.0 Session Objectives

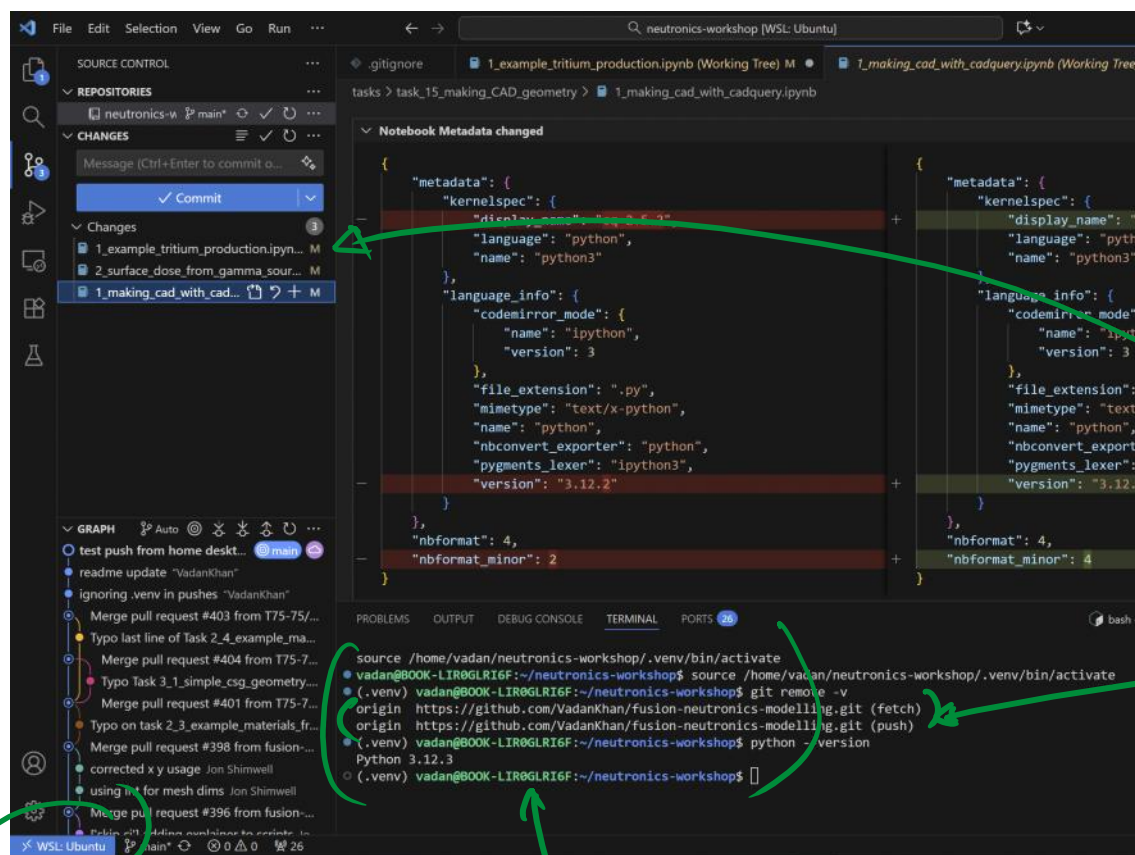
Building upon the successful environment setup in the previous week, the objectives for this session were focused on workflow enhancement, collaboration, and ensuring long-term project stability:

1. Integrate the WSL-based OpenMC environment with Visual Studio Code.
2. Create a dedicated project repository on GitHub and migrate the existing local setup into it for version control.
3. Develop and document a clear, step-by-step guide for reproducing the entire setup process.
4. Validate the setup guide by replicating the environment on a secondary machine (Desktop PC).

2.0 Methodology & Practical Implementation

2.1 VS Code Integration The first task was to connect the VS Code IDE (running on Windows) to the Linux environment inside WSL.

- **Process:** I installed the official "WSL" extension by Microsoft in VS Code. With the Ubuntu terminal open and navigated to the project directory (~/.neutronics-workshop), I ran the command code ..
- **Outcome:** This seamlessly opened the project folder in a new VS Code window. The crucial indicator was the green tab in the bottom-left corner, which read "WSL: Ubuntu", confirming that the IDE was now operating within the Linux filesystem. The integrated terminal in VS Code opened directly into the WSL's bash shell, with the Python virtual environment still active. This was a major breakthrough, creating a powerful, unified development interface.



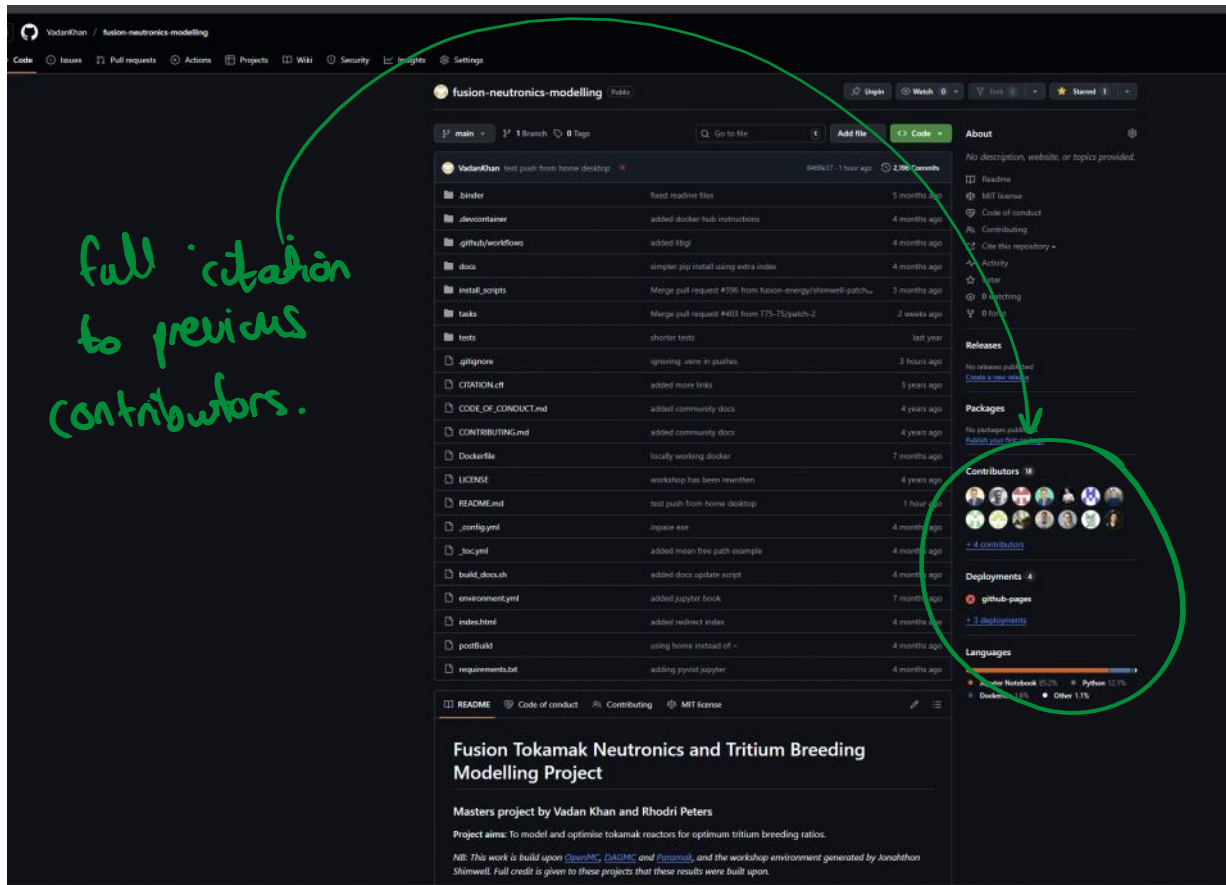
WSL integration to Linux

connected to our github!

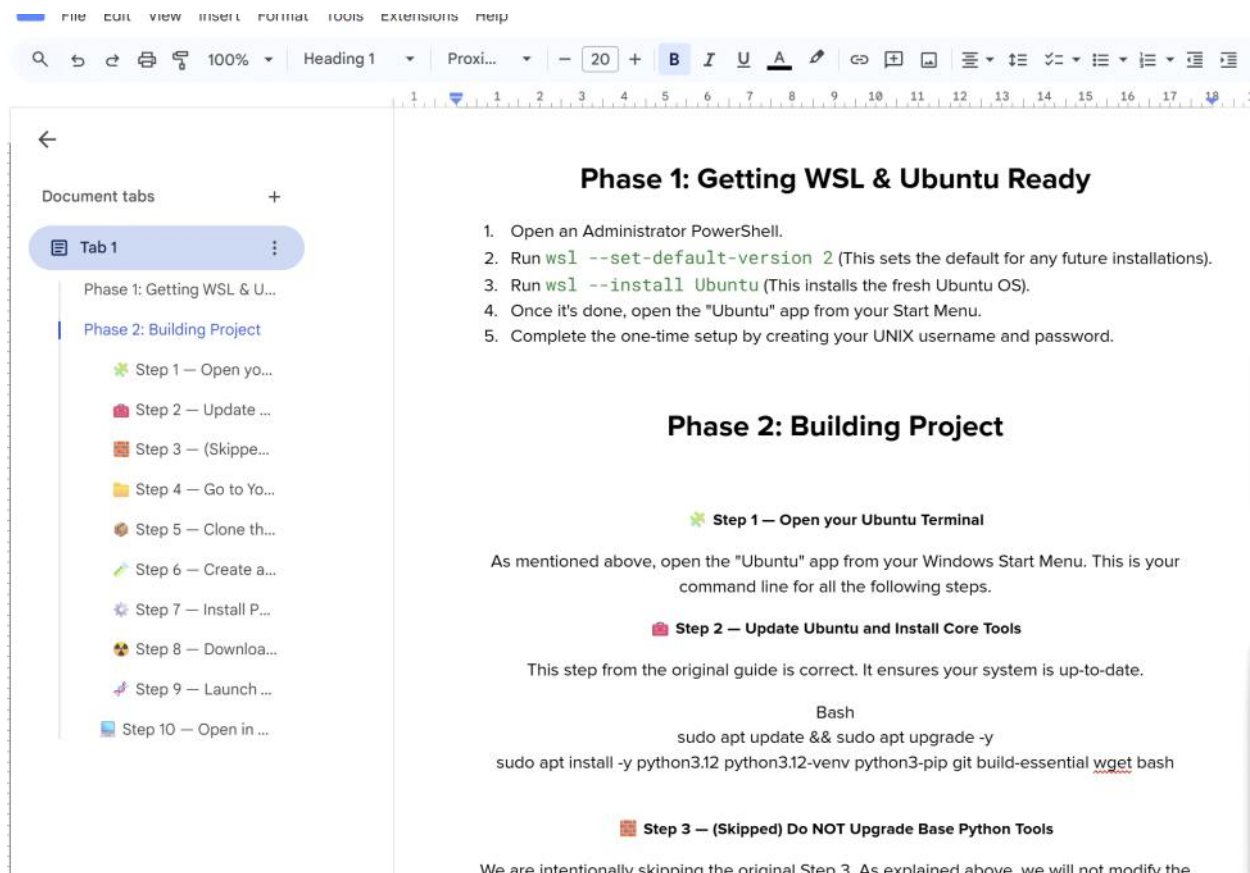
2.2 Establishing a Version Controlled Project Repository To facilitate collaboration and track changes, I created a dedicated repository for our project.

- **Process:** A new repository, fusion-neutronics-modelling, was created on my GitHub account (VadanKhan). On my local machine, I initialised a new Git repository in the project folder, added all relevant files, and made an initial commit. I then linked this local repository to the GitHub remote using git remote add origin ... and pushed the initial build.

- **Outcome:** Our project now has a central, version-controlled home. This decouples our work from the original workshop repository and allows us to manage our specific code, models, and results.



2.3 Documenting and Validating the Setup Process A key part of robust scientific work is reproducibility. I decided to perform a full re-installation from a clean slate, documenting each step precisely to create a guide for my lab partner and for my future self.



3.0 Challenges Encountered & Solutions: WSL Environment Corruption

I then decided to apply this guide to follow the entire setup process from scratch, onto my personal desktop PC.

- Setting up wsl
- Setting up OS, git and python
- Installing project **but from our own repo instead**
- Building environment
- Linking to the github repo
- Successful operation of code.

This will act as another failsafe machine to work on. Is also a better machine to use, with better processing power and preferred setup to code from.

3.1 Challenges Encountered & Solutions: WSL Environment Corruption

During the attempt to setup on desktop pc, the existing WSL Ubuntu distribution became corrupted.

- **Problem:** Upon launching the Ubuntu application, the terminal was completely blank and unresponsive. Standard WSL commands in PowerShell were also failing or hanging indefinitely. The `wsl --unregister Ubuntu` command, intended to remove the faulty distribution, would freeze on "Unregistering...".
- **Troubleshooting:** My troubleshooting process was systematic:
 1. Attempted to forcefully shut down the WSL service with `wsl --shutdown`. This command also hung.
 2. Escalated to using Windows Task Manager to terminate the underlying WSL virtual machine process (vmem). This was denied due to system permissions.
 3. The final resort was a full system reboot, which successfully terminated all stuck processes.
- **Solution:** After the reboot, I was able to successfully uninstall **via uninstalling Ubuntu via "apps and features"**.
- **Recovery:** I then re-installed from scratch using a single command in an administrative PowerShell: `wsl --install`. This modern command streamlined the process, enabling all features and installing a fresh Ubuntu distribution. From there, I followed my newly drafted setup guide.

While frustrating, this failure was incredibly valuable. It forced me to learn the deep mechanics of WSL management and resulted in a much more robust and thoroughly tested setup guide.

4.0 Results & Key Milestones

This session was highly productive, despite the technical challenges:

- The development workflow has been massively improved through successful VS Code and WSL integration.
- **The project is now securely managed under version control in its own dedicated GitHub repository.**
 - Link to repository: <https://github.com/VadanKhan/fusion-neutronics-modelling/tree/main>
- A complete, validated, and stress-tested guide for setting up the OpenMC environment has been produced.
- The entire environment was successfully replicated on my secondary PC, proving the guide's effectiveness and enabling flexible working. I can now develop, commit, and push changes from either machine.

5.0 Next Steps

With the foundational software engineering and environment setup now complete and stable, the project can pivot entirely to the research objectives. The immediate plan for the next session is to:

- 1. Begin scripting the first simple breeder blanket model using the OpenMC Python API.**
 1. Run the first simulation to calculate a baseline neutron flux spectrum and an initial TBR value.
- 2. Focus on setting up write up tools and begin report**
- 3. Attempt setting up on MacOS**

Supervisor Meeting

Literature Review:

- Do as much literature review as there is literature available
- Feel free to add some basic knowledge, so the reader starts at a point of agreement. Sets a good tone

Methodology:

- Don't focus on the very specific implementations
- General Explanations of Monte Carlo Methods

Modelling Planning:

- Start at basic models from Shimwell and Tutorials
- Start at default paramak geometry, compare to literature
- Materials list:
 - Start from simpler compositions, then evolve to multi species
 - Start at Enrichment sweeps
 - Start lead proportion sweeps

Formatting:

- 2 columns is standard,
- 2 columns also allows for more words
- Either referencing style

Extra Thursday Session:

Subject: LaTeX Environment Setup & OpenMC Installation on ARM64 Architecture

1. Productivity Tools: LaTeX Environment Setup

Today's initial focus was on establishing an efficient workflow for writing and documentation. I successfully configured a professional LaTeX environment using **Visual Studio Code**.

- **Editor:** VS Code with the "**LaTeX Workshop**" extension.
- **Compiler/Distribution:** **MiKTeX** was installed to handle the core TeX/LaTeX compilation.
- **Dependencies:** **Strawberry Perl** was also required for full functionality on Windows.

The setup is highly effective. A key benefit is the "sync-to-PDF" feature, which allows for seamless navigation between the source .tex code and the compiled PDF output, greatly speeding up the editing and review process.

2. Collaboration and Project Planning

Upon arriving on campus, I met with my colleague, Rhodri. I shared the details of my new LaTeX setup with him. We then proceeded to discuss the database values we have recently obtained, outlining a clear plan for our upcoming simulation work.

3. Technical Challenge: Installing Simulation Environment on macOS (ARM64)

A significant portion of the day was dedicated to installing our primary simulation software, **OpenMC**, onto Rhodri's ARM64 (Apple Silicon) MacBook. This proved to be a major technical challenge.

- **Initial Attempt (Standard pip install):** The standard installation via `pip install openmc` failed immediately. The root cause is that the official package repository (PyPI) does not host pre-compiled binaries ("wheels") for the arm64 architecture.
- **Advanced Attempt (Building from Source):** This required moving beyond standard package management and attempting to compile the entire C++ software from its source code—a valuable new skill for me.
 - **Direct Pip Build:** Attempts to use `pip install .` directly within the cloned repository also failed, as pip could not successfully build the C++ dependencies on its own. This did, however, demonstrate how the wheel build process works.
 - **Manual CMake Build:** We then undertook a full manual compilation using CMake. This involved several key steps:
 1. Identifying the correct compilers (gcc-15, g++-15) installed via Homebrew, as the default Apple Clang compiler lacks required OpenMP support.
 2. Configuring the build with CMake to point to these specific compilers.
 3. Running `make` to compile the C++ source code.
 4. Finally, running `pip install .` within the activated virtual environment to install the Python bindings and link them to the newly compiled C++ libraries.

4. Conclusion and Next Steps: No MacOS arm64 openMC!

Despite the C++ compilation appearing to complete successfully, the final Python integration failed. Though `import openmc` worked within the virtual environment, **Python returned an error when running the cells "could not find openmc"**.

- This is the same error as when building via docker on week 1.

After several hours of exhaustive troubleshooting, it is almost certain that the **OpenMC software is not compatible with this hardware architecture out-of-the-box**. While a workaround might theoretically exist, it would likely require deep modifications to the source code or build process, which is beyond the scope of this project and is not supported by the OpenMC developers.

Path Forward: We cannot rely on this machine for our simulations. We will need to devise a workaround, such as using a university-provided Linux machine, a cloud computing environment, or my own laptop for the simulation tasks.

Link to repository: <https://github.com/VadanKhan/fusion-neutronics-modelling/tree/main>

- Experimented code lies within tasks
- Commit 8469e37, 8469e373f5f1d026ee0a0c929f3910fdea1609a8

Link to Drive of Shared Documents:

<https://drive.google.com/drive/u/0/folders/19LqvtoSB7r2HaBN0A3UFINr3q7Z7n65p>

1.0 Overview

This week's activity was dedicated to establishing and refining the project's documentation and data processing infrastructure, which runs in parallel with the primary simulation tasks. A significant portion of time was allocated to report writing to ensure project documentation keeps pace with technical development. Further efforts involved enhancing the LaTeX report framework, particularly bibliography management, and improving the workflow for acquiring nuclear data for analysis.

<https://www.nndc.bnl.gov/ndf/extended.html>

<https://www.nds.iaea.org/exfor/servlet/E4sSearch2>

2.0 Project Report & LaTeX Typesetting

With the initial simulations established, dedicated time was required to formally document the project's background, methodology, and initial findings in the master's project report.

2.1 Report Formatting and Structure

Initial experimentation was conducted on the report's LaTeX document structure to improve readability and presentation. The document formatting was adjusted from a two-column format, common for short papers, to a single-column layout. This is more appropriate for a detailed, chapter-based report or thesis, allowing for better placement of figures and tables. The first figures, likely conceptual diagrams or initial data plots, were successfully integrated using the figure environment.

2.2 Bibliography Management

A robust and scalable referencing system is critical for the project report.

- **Reference Collation:** The Zotero reference management software is being used to collect, organize, and annotate all papers, technical reports, and data sources.
- **LaTeX Integration:** We explored migrating the bibliography backend from the traditional bibtex to the more modern biblatex.
 - **Objective:** biblatex offers superior flexibility in citation styles, better handling of non-ASCII characters (e.g., author names), and more logical separation of style from content.
 - **Outcome:** Initial attempts to compile the document using biblatex failed, likely due to package conflicts or incorrect backend configuration.
 - **Status:** To maintain progress on the report, the bibliography has been temporarily reverted to a standard bibtex implementation. Resolving the biblatex compilation failure remains a low-priority technical task.
 - Want to look at biblatex again in the future

3.0 Nuclear Data Processing Workflow

The material selection process described in the previous lab entry (Entry 1, Section 3.1) was based on analyzing cross-section data. This week, the workflow for acquiring this data was significantly improved.

- **Previous Method:** Relied on web-based plotting tools provided by nuclear data services (e.g., ENDF). This is fast for visualization but does not provide the raw data needed for custom analysis and publication-quality plotting.
- **New Method:** Developed a workflow to programmatically query and download cross-section data tables directly from the Evaluated Nuclear Data File (ENDF).
- **Advantage:** This provides full control over the data. We can now use Python (e.g., matplotlib) to process the raw data and generate high-quality, customized graphs comparing the relevant reaction cross-sections (e.g., $\sigma_{\text{f}}(\text{E}), \sigma_{\text{scat}}(\text{E}), \sigma_{\text{tot}}(\text{E})$) for all candidate materials on a single, clear set of axes.

4.0 Summary and Next Steps

This week's focus on documentation and data tooling provides the necessary support structure for the project's simulation goals. While simulation work was paused, these tasks are essential for the final project submission.

The immediate priorities are now to:

1. **Return to the primary technical bottleneck:** Resolve the catdogmrc Python library conversion failure (from Entry 1, Section 2.3). This remains the highest-priority simulation task.
2. **Apply the new ENDF data workflow:** Generate the formal cross-section plots for the material selection (Li, He, Zr, Be) to be included in the project report.
3. **Continue simulation:** Once the catdogmrc converter is functional, proceed with integrating the Paramak geometries into the OpenMC simulation.

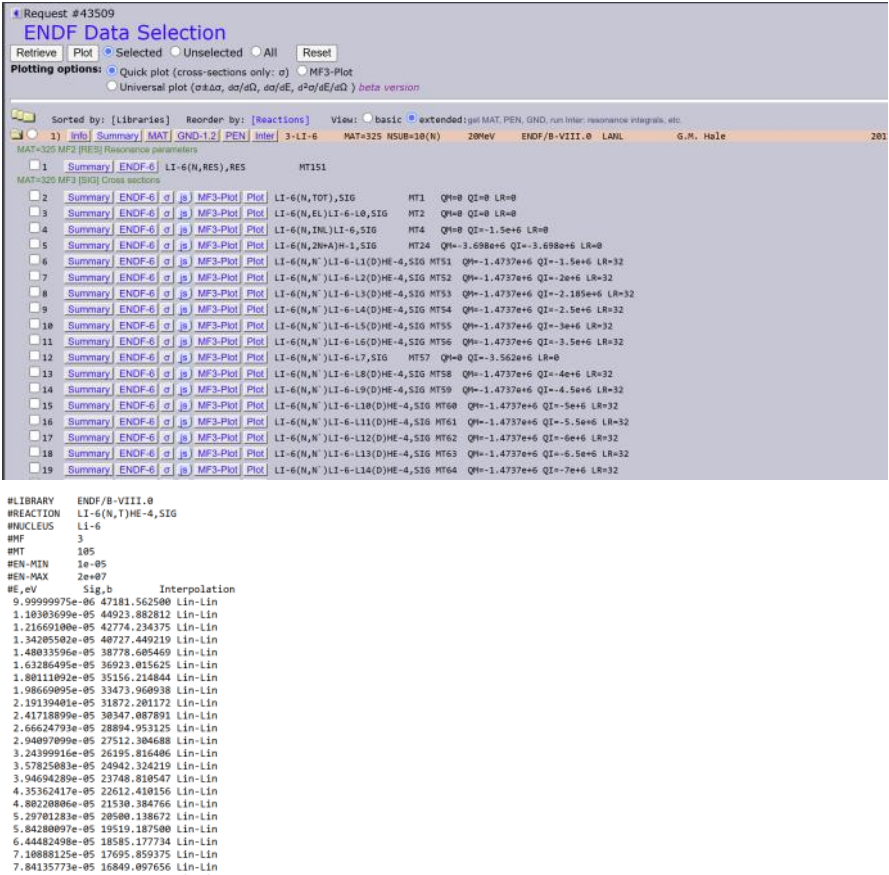
Link to repository: <https://github.com/VadanKhan/fusion-neutronics-modelling/tree/main>

Link to Drive of Shared Documents:

<https://drive.google.com/drive/u/0/folders/19Lqvt0SB7r2Ha8NOA3UFINr3q7Z7n65p>

Relative Path of Thesis:

- Root Masters Directory (personal onedrive) \ Thesis



1.0 Introduction

The primary objective of this project phase is to establish a robust and flexible computational workflow for conducting fusion neutronics analysis, specifically focusing on Tritium Breeding Ratio (TBR) calculations, using the OpenMC Monte Carlo transport code. A critical component of this workflow is the geometry generation pipeline. This entry details the initial exploration of different geometry generation methods and the successful establishment of a baseline TBR simulation using OpenMC's native capabilities.



2.0 Exploration of Geometry Generation Methods

A realistic simulation of a fusion reactor (e.g., a tokamak) is critically dependent on an accurate geometric representation of its complex components. We investigated several potential workflows to translate engineering designs into a format compatible with OpenMC.

2.1 OpenMC Native Geometry (CSG)

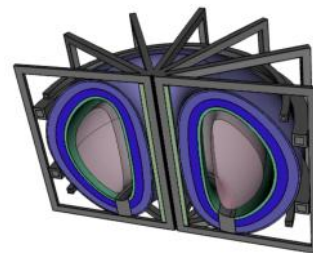
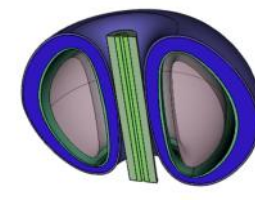
Initial experiments were conducted using OpenMC's native geometry builder. This system relies on Constructive Solid Geometry (CSG), defining volumes through the Boolean operations (union, intersection, complement) of simple geometric primitives (planes, spheres, cylinders, etc.).

- **Findings:** This method is effective for constructing simple, idealized models. It provides a rapid way to test physics setups (materials, sources, tallies) without the overhead of external tools.
- **Limitations:** The CSG approach is manually intensive and impractical for representing the complex, curved, and detailed components of an engineering-grade CAD (Computer-Aided Design) model.

2.2 CAD-based Parametric Workflows

For realistic models, a CAD-based workflow is essential. We explored a pure-Python pipeline leveraging Cadquery and Paramak.

- **Cadquery:** This is a Python-based, open-source CAD kernel. Its key advantage is the ability to programmatically and parametrically define 3D models. This is ideal for our purposes, as it allows geometry to be scripted, version-controlled, and iterated upon for optimization studies.
- **Paramak:** This is a high-level Python package specifically designed for generating parametric fusion reactor geometries. It is built upon Cadquery and simplifies the creation of complex tokamak and stellarator assemblies from a minimal set of key physics parameters.
 - **Experiments:** We successfully generated several Cadquery assemblies using Paramak, including:
 - A SimpleSphericalTokamak model.
 - A more detailed, AnnotatedTokamak model with named components.
 - A TokamakWithCoils model, representing a higher level of engineering detail.



Summary Explanation of model and linking

- CADquery generates CAD models (can be exported as .step files)
 - Paramak is a subpackage of cadquery, that can quickly generate parameterised fusion geometries as Cadquery assemblies with a single line python input.
- DAGMC models are models specifically used for MC modelling, and openMC is compatible with it
- To use CAD in openMC, we must use the CAD to DAGMC converter to get the geometry in a compatible format

2.3 The CAD-to-Monte-Carlo Bridge: DAGMC

OpenMC, like many Monte Carlo codes, cannot use CAD files (e.g., .step files exported by Cadquery) directly. The standard method for incorporating such models is the Direct Accelerated Geometry Monte Carlo (DAGMC) workflow.

- **Process:** This workflow requires a two-step process:
 1. **CAD Generation:** A 3D model is created in a CAD tool (like Cadquery/Paramak).
 2. **Conversion:** The CAD model is converted into a faceted, watertight geometry format compatible with DAGMC (e.g., an .h5m file). This converted model can then be loaded directly by OpenMC.
- **Identified Bottleneck:** The critical link in this chain is the conversion tool. We investigated the cadtodagmc Python library for this purpose. Initial attempts to utilize this converter in our environment failed. **Resolving this conversion issue is now the primary bottleneck and highest-priority task**, as it is essential for integrating our Paramak-generated geometries with OpenMC.

```
from cad_to_dagmc import CadToDagmc

OSError                                Traceback (most recent call last)
Cell In[1], line 1
----> 1 from cad_to_dagmc import CadToDagmc

File ~/neutronics-workshop/.venv/lib/python3.12/site-packages/cad_to_dagmc/_init_.py:16
    12 __version__ = get_version(root=".", relative_to=__file__)
    14 __all__ = ["__version__"]
--> 16 from .core import *

File ~/neutronics-workshop/.venv/lib/python3.12/site-packages/cad_to_dagmc/core.py:4
     2 from typing import Iterable
     3 import cadquery as cq
```

```
OSError                                Traceback (most recent call last)
Cell In[1], line 1
----> 1 from cad_to_dagmc import CadToDagmc

File ~/neutronics-workshop/.venv/lib/python3.12/site-packages/cad_to_dagmc/_init_.py:16
    12 __version__ = get_version(root=".", relative_to=__file__)
    14 __all__ = ["__version__"]
--> 16 from .core import *

File ~/neutronics-workshop/.venv/lib/python3.12/site-packages/cad_to_dagmc/core.py:4
     2 from typing import Iterable
     3 import cadquery as cq
     4 import gmsh
     5 import numpy as np
     6 from cadquery import importers

File ~/neutronics-workshop/.venv/lib/python3.12/site-packages/gmsh.py:92
    87 if not libpath:
    88     print("Warning: could not find Gmsh shared library " + libname +
    89           " with ctypes.util.find_library() or in the following
    90 locations: " +
    91       str(possible_libpaths))
--> 92 lib = CDLL(libpath)
    94 try_numpy = True # set this to False to never use numpy
    96 use_numpy = False

...
--> 379 self._handle = _dlopen(self._name, mode)
    380 else:
    381     self._handle = handle
OSError: libGLU.so.1: cannot open shared object file: No such file or directory
```

```
--> 379 self._handle = _diopen(self._name, mode)
380 else:
381     self._handle = handle
OSError: libGLU.so.1: cannot open shared object file: No such file or
directory
```

3.0 First Tritium Breeding Simulation (CSG Model)

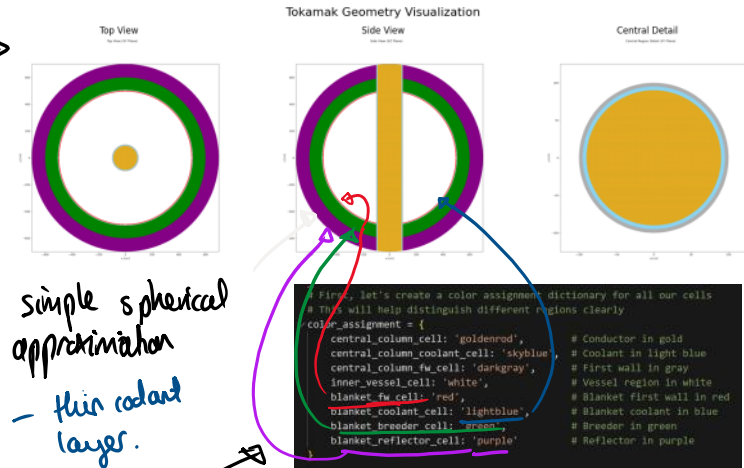
While the CAD-to-DAGMC pipeline is under development, a parallel effort was undertaken to establish a complete, "closed-loop" simulation framework for TBR analysis. This was achieved using a **simplified, tokamak-like geometry built with the native OpenMC CSG builder** (adapted from the Shimwell neutronics workshop optimization final task). This baseline model, while geometrically simple, is invaluable for developing and verifying the full physics setup and scripting for optimization.

- The model has been setup with a tally to run a **basic first TBR simulation**, a critical first step:

- A particle filter and Tally were implemented to score the tritium-producing reactions within the breeder blanket.

- We first iterated the set of materials for the

- Breeder
 - First wall
 - Coolant
 - Reflector (last wall material of the tokamak)



- We found an optimum set of materials after plotting the cross sections for scattering and absorption for each of these materials, a key step. **Crucially we split apart the plotting for Scattering and absorption, as we would want to minimise or maximise these depending on the component.**

- **Breeder Blanket: Liquid Lithium (Li).** Required for the tritium-producing reaction.
 - **Coolant: Helium (He).** Chosen for its chemical inertness and exceptionally low neutron absorption cross-section, minimizing parasitic neutron loss.
 - **First Wall: Zirconium (Zr).** Chosen as a representative structural material with acceptable neutronic performance and high-temperature properties.
 - **Reflector: Beryllium (Be).** Placed at the back of the blanket, Beryllium acts as an effective neutron multiplier via the (n, 2n) reaction, reflecting neutrons back into the breeding blanket and significantly enhancing the TBR.

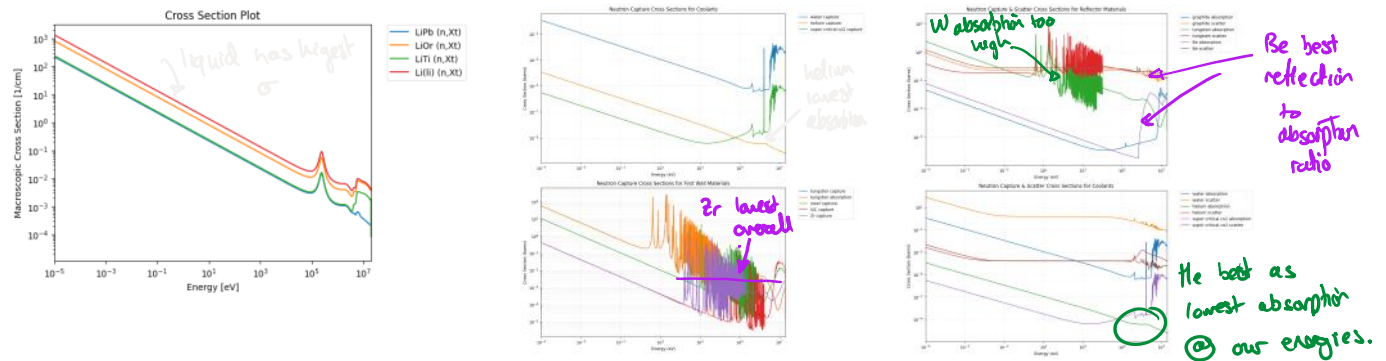
```
mat_breeder = liquidlithium
# 99.9999% 6Li and 0.0001% 7Li is optimal. Too high and not enough neutron multiplication, too low and too much input energy loss!

mat_coolant = helium
# Neutronically helium is best due to low absorption. However it does have poor heat transfer properties (low density)

mat_firstwall = zirconium
# Zirconium seems to be optimal, only low absorption peaks for much lower energies

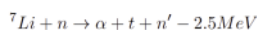
mat_reflector = beryllium
# Beryllium seems to be optimal, similar to graphite, still low absorption but slightly higher reflectivity.

# put in your chosen material names
my_materials = openmc.Materials([mat_conductor, mat_coolant, mat_firstwall, mat_breeder, mat_reflector])
```

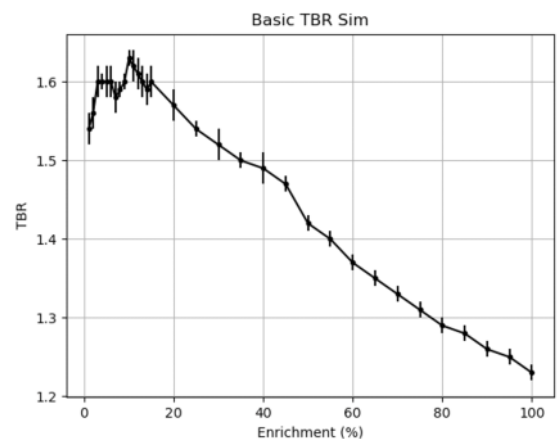


- We then decided to optimise further, and gathered data for a **first plot a sweep of enrichment of lithium6 to lithium7, another key result**

- **Result:** Data was successfully gathered to plot TBR as a function of ^6Li enrichment. This constitutes our first critical optimization result and validates the simulation loop's functionality.
 - **We found that ~ natural enrichment was actually best for liquid lithium!**
 - This is likely due to the fact that liquid lithium has no breeder
 - So ^7Li is actually necessary for providing neutrons and breeding (acting as a multiplier)
 - This likely causes issues on energy production down the line, as this reaction requires energy.



- Would be good to get actual reaction rate numbers split up per element



4.0 Summary and Next Steps

We have successfully established a foundational framework for TBR simulation and optimization. This "closed-loop" system, based on a simplified CSG model, allows us to test and refine our physics setup, material definitions, and analysis scripts.

The main avenue for future development is clear and is our highest priority:

1. **Resolve the cadtodagmc conversion failure.** This will involve debugging the library, checking dependencies, and ensuring a stable conversion process.
2. **Integrate Paramak models.** Once conversion is successful, we will replace the simplified CSG geometry with the Paramak-generated models.
3. **Perform Realistic TBR Analysis.** This will allow us to conduct high-fidelity neutronic studies on realistic, parametric reactor designs.

Meeting Notes

- ☒ - Don't need too many plots, just 3 for first sections is fine
- ☒ - Check library referencing guide for vancouver referencing (full names or not)
- ☒ - \usepackage{mhchem} for chemical formulae
- ☐ - Bullet points, not ideal

Report Checkpoints

- ☐ Get to 10 good pages without references
- ☒ Shift to Biblatex
- ☒ Try adjusting line spacing and font to 12
- ☒ Reduce font size of references, and Fixing referencing semantics
- ☐ Tweak and improve equation referencing
- ☐ Add introduction to Boltzmann and neutron transport.

Link to repository: <https://github.com/VadanKhan/fusion-neutronics-modelling/tree/main>

Link to Drive of Shared Documents:

<https://drive.google.com/drive/u/0/folders/19LqvtoS87r2HaBN0A3UFINr3q7Z7n65p>

Relative Path of Thesis:

- Root Masters Directory (personal onedrive) \ Thesis

Objective

To establish a complete, end-to-end simulation framework for creating reactor models in Paramak, converting them to DAGMC geometry, and running neutronics simulations in OpenMC. The primary output of this week was the geometryworkflow.ipynb notebook detailing this pipeline.

Environment & Collaboration Setup

A major parallel task this week involved resolving significant, long-standing setup issues for a project partner.

- **Problem:** After many weeks of struggling to get a functional OpenMC environment on the partner's local machine, a new approach was needed.
- **Solution:** A stable, remote development environment was established by setting up an **integrated VS Code window** connected directly to a cluster machine via the **SSH remote integration**.
- **Implementation:** This first required successfully setting up the complete simulation code environment on the cluster. This time, **Conda** was used for package management instead of manual virtual environments, which proved successful.
- **Outcome:** This was a significant breakthrough. **The partner can now finally access and run the code, enabling them to contribute.** This success also validates the **portability of the code and its environment** when managed with Conda.

Converting CAD geometry to neutronics

CAD geometry can be converted with cad-to-dagmc. This makes DAGMC geometry which particle transport codes like OpenMC can understand and simulate particle transport through.

In this example we are going to convert some CAD to a neutronics model and simulate particles. Read more about DAGMC geometry here <https://valinn.github.io/DAGMC/>

First import the package used to convert CAD to DAGMC geometry

```
from cad_to_dagmc import CadToDagmc
```

Make a instance of CadToDagmc

```
model = CadToDagmc()
```

Add the step file to the model. This step file contains 6

```
model.add_step_file('...')
```

Now export the DAGMC h5m geometry file that can be used in OpenMC and other codes that support DAGMC. The mesh will be made with default meshing parameters

```
model.export_dagmc_h5m_file(filename='dagmc.h5m')
```

The mesh production was quick but you might want to customize it. For example add extra mesh nodes to a particular solid. The next mesh is customised with the set_size argument to have small mesh (size is 0.5) elements for the first solid and large mesh (size is 2.0) for the other solids.

You can change the numbers used for each solid to specify different mesh sizes for each solid in the geometry.

```
model.export_dagmc_h5m_file(
    set_size=[
        ...
    ]
)
```

The resulting DAGMC.h5m can be used in neutronics simulations.

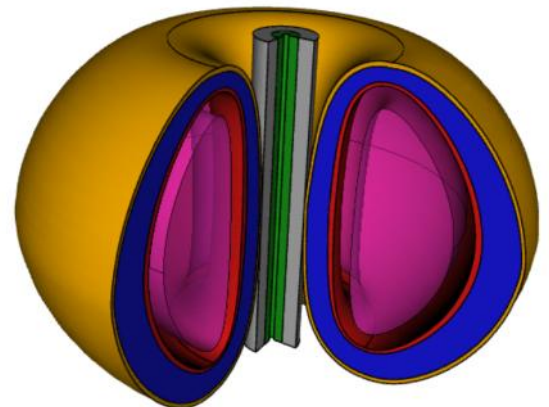
For more details of how to convert CAD geometry see then next task which goes over

- Additional mesh parameters
- 3D volume mesh production

Progress & Activities

1. Environment Setup:

- Identified and implemented a fix for running cadtodagmc on the current operating system. This required installing two libraries:
 - sudo apt-get install libglu1
 - sudo apt-get install libxft2
 - (also always good to run sudo apt-get update first)



1. Paramak Model Development:

- Investigated the deeper implementation of how Paramak builds CadQuery assemblies.
- This included incomplete attempts at recreating custom reactor functions to better understand the underlying logic.
- A 180-degree sector reactor model was successfully created in Paramak as the basis for the workflow.
- **Established a COMPONENT_MAP dictionary to act as a single source of truth for material tags, component names, and colors.**

much easier

Step 3: Define OpenMC Materials

Creates an `openmc.Material` object for each material tag found in the `paramak` model exactly.

```
print("Defining OpenMC materials...")
materials_list = []
for tag in reactor_material_tags:
    # Create a new material with the 'exact' name as the tag
    mat = openmc.Material(name=tag)

    if tag == 'plasma':
        mat.add_element('H', 1.0, 'ao') # Dummy plasma
        mat.set_density('g/cm3', 1.0e-6)
    else:
        # Default for all other solid layers
        mat.add_element('Fe', 1.0, 'ao') # Dummy steel
        mat.set_density('g/cm3', 7.8)

    materials_list.append(mat)

# Create the final Materials collection
my_materials = openmc.Materials(materials_list)
print(f"Created {len(my_materials)} materials")
```

1. Geometry Conversion Pipeline (Paramak -> DAGMC -> OpenMC):

- **Paramak to DAGMC:** Learned the process for converting the Paramak CAD model into a DAGMC .h5m file. This was achieved using the `cad_to_dagmc` library, successfully exporting `dagmc_reactor.h5m`.
- **DAGMC to OpenMC:** Learned how to load the new .h5m file into OpenMC using `openmc.DAGMCUniverse`.
- **Region Tagging & Materials:** A significant effort was dedicated to learning how to correctly tag regions within Paramak and define



-

Index!

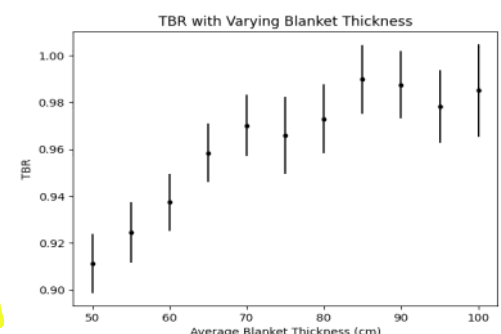
- [illegible]

- Mock Report Check: 2 page summary of the overall report, however loose. Can include snippets of random things you want to check

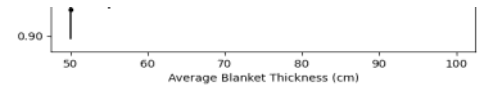
- Labbook: Supervisor
- Report: Primary Assessor and Secondary Assessor
- Interview: Primary Assessor and Supervisor

This week's work successfully established the complete framework to build Paramak reactors, convert them for use in OpenMC with DAGMC, and run a simulation. The `geometryworkflow.ipynb` notebook documents this entire pipeline.

- Formalize the specific reactor design to be used for analysis.
- Formalize the simulation run parameters for specific investigations (e.g., analyzing the impact of breeder blanket thickness).
 - Lab partner actually had a go at this with a basic version of the script (not the full notebook) now that he had access to the code. Got some



- Formalize the simulation run parameters for specific investigations (e.g., analyzing the impact of breeder blanket thickness).
 - Lab partner actually had a go at this with a basic version of the script (not the full notebook) now that he had access to the code. Got some great results!
 - However will need to review the backend and integrate my developments gain a secure set of results for next week



These values are for 60% enriched Li17Pb83 with the basic Paramak model

Integrated Blanket Sweep

Objective:

Refactor the end-to-end simulation notebook into a modular, function-based workflow. This will improve reusability, maintainability, and enable complex parametric studies. Once modularized, use this new workflow to run a parametric sweep of the breeder blanket thickness and its effect on TBR.

Workflow Modularization:

The notebook was successfully refactored into a series of discrete, callable cells/functions, each responsible for one stage of the simulation pipeline:

- Setup & Configuration:** A centralized output_dir (Path.cwd().parent / "outputs") was defined to ensure all artifacts are stored in a single, predictable location.
- Geometry Definition (basic_tokamak):** A function was created to generate the paramak.Reactor object, parameterized by the thickness multiplier a.
- Material Library (materials_library):** A function to define and return all necessary openmc.Material objects based on material tags from the reactor.
- Settings & Tallies (settings_code):** A function to define the simulation settings (batches, particles) and the crucial tbr tally.
- CAD-to-DAGMC (converter_code, geometry_code):** Functions to handle the conversion of the Paramak CAD to a DAGMC .h5m file and define the openmc.DAGMCUniverse.
- Model Assembly:** The main openmc.Model object is constructed by calling the functions above in sequence.
- Plotting & Cleanup:** Specific cleanup logic (e.g., removing stale model.xml files) was implemented to ensure post-simulation plotting correctly uses the DAGMC-generated plots.xml.

Application: Breeder Blanket Thickness Sweep

This new modular structure was immediately leveraged to build the thickness sweep. The sweep loop simply calls the modular functions (basic_tokamak(a=i), settings_code, etc.) for each i in the thickness_multipliers range (50 to 150).

Problem Encountered: The workflow ran perfectly for the first iteration (multiplier = 50) but failed on the second iteration (multiplier = 55).

- Error:** HDF5-DIAG error, Resource temporarily unavailable (errno = 11).
- Root Cause:** The script was not properly closing the statepoint.10.h5 file after reading the TBR results. This left a file lock, which prevented OpenMC from overwriting the file on the next loop.

Solution and Modifications:

A two-part solution was implemented to resolve the file conflict and ensure robust looping:

- Correct File Handling (Primary Fix):** The openmc.StatePoint() call was wrapped in a with statement. This is the standard Pythonic way to manage resources like files. It guarantees that the HDF5 statepoint file is automatically and properly closed (and its lock released) as soon as the data is extracted, before the loop proceeds to the next iteration.
 - Old Code:** sp = openmc.StatePoint(statepoint_file)
 - New Code:** with openmc.StatePoint(statepoint_file) as sp:
- Aggressive File Cleanup:** As a secondary precaution, an explicit cleanup step was added to the beginning of each loop. This uses pathlib to find and remove all simulation files from the previous run within the outputs/ directory. This ensures each iteration starts in a clean environment.
 - Files Targeted:** *.h5, *.h5m, *.xml, *.out
 - Implementation:** pathlib.Path.glob() and file.unlink() inside the loop.

```
def basic_tokamak(paramak_colors, a = 100, t = 0.55):
    paramak_colors = {
        name: details[0] for name, details in COMPONENT_MAP.items()
    }

    my_reactor, reactor_material_tags, radial_build_list = basic_tokamak(paramak_colors)

    # Get the material tags from the reactor model.
    reactor_material_tags = my_reactor.names()
    print(f"Reactor material tags found: {reactor_material_tags}")

    # --- USER REQUEST: Print attributes of my_reactor ---
    print(f"Attributes of my_reactor object:\n{dir(my_reactor)}\n")

    # This line will try to display the reactor object in Jupyter
    my_reactor
```

Step 2: Convert CAD to DAGMC

Uses cad_to_dagmc to convert the in-memory paramak object into a .h5m file for DAGMC.

```
def converter_code(my_reactor, reactor_material_tags):
    h5m_filename = converter_code(my_reactor, reactor_material_tags)
```

Step 3: Define OpenMC Materials

Creates an openmc.Material object for each material tag found in the paramak model. The material name must match the tag exactly.

```
def materials_library(reactor_material_tags):
    my_materials, materials_list = materials_library(reactor_material_tags)
    print(my_materials)
```

```
print(f"--- Starting Breeder Blanket Thickness Sweep ---")
print(f"All simulation outputs will be written to: {output_dir}")

avg_thickness = 75 # blanket thickness in centimeters, standard value of 75cm
triangularity = 0.55 # triangularity, standard value of 0.55

thickness_multipliers = np.arange(50, 151, 5)
thickness_values = thickness_multipliers * avg_thickness/100
tbr_values = []
tbr_errors = []

for i in thickness_multipliers:
    print(f"\n--- Running simulation for thickness multiplier: {i} ---")

    # --- NEW: Aggressive cleanup of old files ---
    # This deletes files from the "previous" iteration to prevent conflicts,
    # as you requested.
    print(f"Cleaning output directory: {output_dir}")
    extensions_to_remove = ['*.h5', '*.h5m', '*.xml', '*.out']
    files_removed = 0
    for ext in extensions_to_remove:
        for f in output_dir.glob(ext):
            try:
                f.unlink()
                files_removed += 1
            except OSError as e:
                print(f"Warning: Could not remove {f.name}. Error: {e}")
    print(f"Removed {files_removed} old files.")
    # --- END OF CLEANUP ---

    # --- 1. Model Setup (Unchanged) ---
    reactor, material_tags, radial_build_list = basic_tokamak(paramak_colors, a=i)
    settings_i, tallies_i = settings_code(radial_build_list)
    materials, materials_list = materials_library(material_tags)

    my_model = openmc.Model(
        geometry=geometry_code(converter_code(reactor, material_tags)),
        materials=materials_list,
        settings=settings_i,
        tallies=tallies_i
    )

    # --- 2. Run simulation in the correct output directory (Unchanged) ---
    print(f"Running OpenMC in: {output_dir}")
    statepoint_file = my_model.run(cwd=output_dir)
    print(f"Statepoint file created: {statepoint_file}")
```

starts in a clean environment.

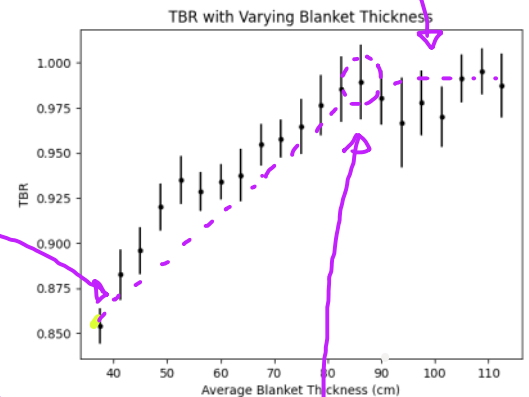
- **Files Targeted:** *.h5, *.h5m, *.xml, *.out
- **Implementation:** `pathlib.Path.glob()` and `file.unlink()` inside the loop.

Results:

The robust, modular workflow successfully completed the entire breeder blanket thickness sweep. The file lock and cleanup issues were fully resolved. The `tbr_values` and `tbr_errors` were collected and plotted against the `thickness_multipliers`.

The results clearly show the expected correlation between blanket thickness and tritium breeding performance:

- **Positive Trend:** The TBR increases as the blanket thickness multiplier increases, starting at ~0.857 for a multiplier of 50.
- **Breakeven Point:** Tritium breakeven (TBR = 1.0) is achieved at a thickness multiplier of approximately 85.
- **Saturation:** The rate of TBR increase slows significantly after a multiplier of ~125, indicating diminishing returns. Increasing the thickness from 125 to 150 yields only a minor gain in TBR.
- **Peak TBR:** The maximum TBR achieved in this sweep was approximately 1.1 (at a multiplier of 150).



Next Steps:

- Deeper analysis onto simulation setup validity
- Formalize the specific reactor design to be used for analysis.
- Formalize the simulation run parameters for specific investigations

Report Checkpoints

- ☒ Get to 10 good pages without references
 - ☒ (got to 11+)
- ☒ Add introduction to Boltzmann and neutron transport.
- ☒ Add Tables for Results
- ☒ Tweak and improve equation referencing
- ☐ Add more basic theory introductions (cross section)

Workshop Notes

- Add more justification to all the actions you made / statements you make
- Primary / Secondary / Tertiary (Classical References)
- Check References
- Try Read Aloud Feature
- Try to make a story / narrative
 - Don't talk about your failures, talk about successes
- Need to start writing labbook / report while working
- Good method (for planning) is to write a really granular contents page
 - Subsections, subsubsections, subsubsubsections.
- Be mindful about error bars
- Inkscape for figure generation

Link to repository: <https://github.com/VadanKhan/fusion-neutronics-modelling/tree/main>

- Commit 935d5ad
- 935d5adabdbc91c18f501d6dd46152e55340d67a
- Pipeline under "geometryworkflow.py"

Link to Drive of Shared Documents:

<https://drive.google.com/drive/u/0/folders/19LqvtoSB7r2HaBN0A3UFINr3q7Z7n65p>

Relative Path of Thesis:

- Root Masters Directory (personal onedrive) \ Thesis

Labbook Week 7

11 November 2025 10:43

Tasks

- ✓ Establish separate git branches (so we can easily see live changes)
- ✓ Establish "Base Model" Parameters
- ✓ Confirm Simulation Setup
- ✓ Physically understand / confirm prototype results graphs
 - ✓ Enrichment graphs
 - ✓ Breeder blanket thickness graphs
 - ✓ Triangularity graphs
 - ✓ Coolant Proportion graphs
- ✓ (spend majority of time catching up on report)

Base Model Parameters

- Base Li6 Enrichment
- Thicknesses
- Material choices
 - Breeder: Li17Pb86 - Liquid
 - Zirconium: First Wall
 - Coolant: Water (less common than He, but more available and better TBR performance)
 - Base Structures: Fe (Look at Eurofer but tagging issues with Paramak prevent this)
- Geometry: paramak.tokamak
- Source:
 - Ring source
 - Muir Distribution: 14.08MeV centred, $kt = 20\text{keV}$, $m_{\text{rat}} = 5$
- Settings: "could go higher as runtime bottlenecked by geometry building (~30secs), no. particles (~5 secs)
 - 25 batches of 5000 particles (errors negligible)
- Blended Coolant layers:
 - Liquid: ~ Literature suggests ~ 30% coolant
 - Justification: Based on piping volume estimates for the CN-HCCB test blanket module.
 - Ref: Wang et al. (2019) & Hirose et al. (2024).
 - Solid: ~ Literature suggests ~ 63% packing factor for pebbles (breeder + multiplier material), ~37% coolant.
 - Justification: Standard inter-particle void space for ceramic breeder beds.
 - Ref: Knitter et al. (2013) "Recent developments of solid breeder fabrication".
- BIG APPROXIMATION: we are going to use a **homogenous assumption** for the coolant structures in the breeder blanket
 - Typically coolant piping has more complex structures
 - We are going to uniformly distribute this coolant across the breeder blanket, based on literature results of what volume of the breeder layer is typically filled with coolant
 - We are mindful of risks with this approach, and how it could reduce accuracy (eg. Compare heterogeneous fission reactors with homogenous fission reactors simulations). However given that this is not critically, and that the neutron source is outside the breeder, we will leave this assumption for this semester.
- Additional Literature Info:
 - Neutron Multiplier Fractions:
 - Either Simple liquid mixture (eutectic, lower overall melting point): eg. Li17Pb87
 - Being eutectic is useful to melt more easily
 - Simple molecular compound liquid mixture: eg LiF - BeF₂ (known as FLiBe)
 - Solid: Ionic Compounds of breeder with ionic compounds of Multiplier
 - Separate pebbles for each
 - Breeder Eg: Li4SiO4 (lithium orthosilicate)
 - Multiplier Eg: Be12Ti

- Hirose T, Guan W, Katagiri T, Wakasa A, Someya Y, Nakajima M, et al. *Functional tests for water cooled ceramic breeder blanket system using full-scale mockups*. Fusion Engineering and Design. 2024 Mar; 200:1-6.
- Wang X, Feng K, Chen Y, Zhang L, Feng Y, Wu X, et al. *Current design and R&D progress of the Chinese helium cooled ceramic breeder test blanket system*. Nuclear Fusion. 2019 Jul 1; 59:1-8.

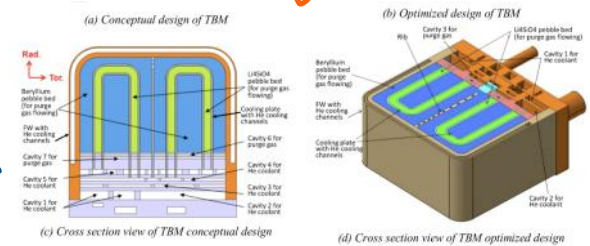
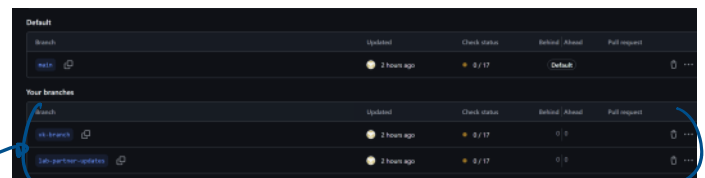


Figure 2. Design optimization of the TBM.

Knitter R, Chaudhuri P, Feng Y, Hoshino T, Yu IK. *Recent developments of solid breeder fabrication*. Journal of Nuclear Materials. 2013 Nov 442:S420-S424.

Git Branches Setup

- Command to move to new branch
git checkout <branch-name>
- Command to generate a new branch:
 - git checkout -b <new-branch-name>
- Command to merge changes from **source branch** into a **target branch**
 - Ensure you move to the target branch first
 - git merge <source-branch-name>



- Was successfully able to setup two new branches for each of us to make changes into, and that we can view easily
- We will now use these branches to publish code updates, so we don't have to manually send each other code.

Simulation Setup Confirmation

- Small tweaks to initial function readability
- Big ? on mesh size (shimwell uses min 2, we used min 5), however runtime limited
 - Min mesh size 5:
 - 36s to build geometry
 - 53s to run single simulation, TBR: 1.43 ± 0.005
 - Min mesh size 2:
 - 34s to build geometry
 - 56s to run single simulation, TBR: 1.43 ± 0.005
 - Ok we will update to use min mesh size of 2 just for fidelity
- Plasma Density: adjusted from 1x10⁻⁶ to 1x10⁻⁹
 - (negligible effect)
 - @article{article, author = {Greenwald, M. and Terry, J. and Wolfe, S.M. and Ejima, S. and Bell, M.G. and Kaye, S.M. and Neilson, G.H.}, year = {2011}, month = {01}, pages = {2199}, title = {New look at density limits in tokamaks}, volume = {28}, journal = {Nuclear Fusion}, doi = {10.1088/0029-5515/51/2/009} }
 - Typically ~ 10²⁰ particles per m³
 - ≈ 10⁻⁹ g/cm³
- Material setup
 - Using homogenous Coolant blend: 30%, water
 - Coolant density (set by temperature/pressure from nmm.material): T=600 but P=15.5MPa (so definitely still liquid)
- Geometry
 - Only defines 100% of the geometry

```
elif tag == 'layer_4':
    breeder = openmc.Material()
    breeder.add_element(
        'Li',
        0.17,
        percent_type='ao',
        enrichment=e,
        enrichment_target='Li6',
        enrichment_type='ao')
    breeder.add_element('Pb', 0.83, 'ao')
    breeder.set_density('g/cm3', 11)
    coolant_he = nmm.Material.from_library(
        'He',
        temperature=800,
        pressure=8000000
    ).openmc_material
    coolant_h2o = nmm.Material.from_library(
        'H2O',
        temperature=600,
        pressure=15500000
    ).openmc_material
    coolant_n2 = nmm.Material.from_library(
        'nitrogen',
        temperature=600,
        pressure=1000000
    ).openmc_material
    enriched_water = openmc.Material()
    enriched_water.add_nuclide('H1', 2, 'ao')
    enriched_water.add_nuclide('O16', 1, 'ao')
    enriched_water.set_density('g/cm3', 0.6611374655967136)
    mat = openmc.Material.mix_materials(
        materials=[breeder, coolant_h2o, fracs=[1-c, c], percent_type='vo', name = 'layer_4'

print(f"Running OpenMC in: {output_dir}")
statepoint_file = my_model.run(cwd=output_dir)
print(f"Statepoint file created: {statepoint_file}")
```

Material Setup

- Using homogenous Coolant blend: 30% water
- Coolant density (set by temperature/pressure from nmm.material): T=600 but P=15.5MPa (so definitely still liquid)
- Geometry
 - Only defining 180° slice to save computation time
 - Roughly at limit of computation time
 - Errors to global parameters are likely <1%, local parameters could vary more however
 - Potential avenue to improve: build full model with no slicing. However this will likely have negligible effects
 - Or to explore making model smaller for runtime
- Settings
 - Neutron Ring source at major radius
 - Potential avenue to improve: (could be developed using openmc_plasma_source), could develop to include a full plasma source of neutrons as opposed to single ring.
 - Example in task 4, 3_plasma_source_plots.ipynb
 - Standard settings from above
- Run:
 - Runs in outputs folder to handle files cleanly and remove files while running
 - Calls modularised functions separately, so not affected by cached data
 - Just running using clean openmc api and getting then getting the tbr tallies from the statepoint file...

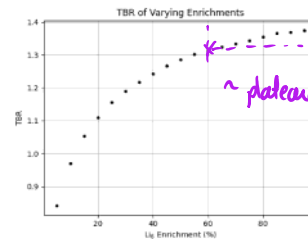
```
print(f"Running OpenMC in: {output_dir}")
statepoint_file = my_model.run(cwd=output_dir)
print(f"Statepoint file created: {statepoint_file}")
# --- 3. Load statepoint file using a 'with' statement ---
#
# THIS IS THE CRITICAL FIX.
#
# Using 'with' ensures the statepoint file is properly closed
# and the file lock is released *before* the loop continues
# to the next iteration.
print(f>Loading statepoint: {statepoint_file} to extract results...")
with openmc.StatePoint(statepoint_file) as sp:
    tbr_cell_tally = sp.get_tally(name='tbr')
    tbr_mean = tbr_cell_tally.mean.sum()
    tbr_std_dev = tbr_cell_tally.std_dev.sum()
    tbr_values.append(tbr_mean)
    tbr_errors.append(tbr_std_dev)
# By this point, the statepoint.10.h5 file is closed.
print(f"TBR: {tbr_mean:.4f} +/- {tbr_std_dev:.4f}")
print("--- ITERATION COMPLETED ---\n\n")
df = tbr_cell_tally.get_pandas_dataframe()
```

First Physics Trends

By increasing the no. neutrons and batches (to settings detailed above) we were able to make the error bars negligible and so observe some really interesting trends!

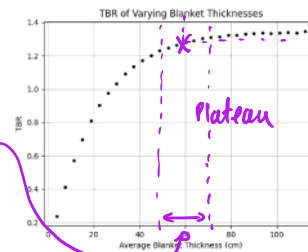
Li6 Enrichment Sweep

- Increasing Li6 Enrichment (for the Li17Pb87) unsurprisingly increased TBR significantly
 - Unsurprising as Li6 Tritium breeding reaction has a much higher cross section and is exothermic (accessible at all neutron energies)
- Seemed to follow a shallow ~ root relationship
- TBR seemed to plateau at ~ 60%,
 - Which matches with literature common rhetoric well



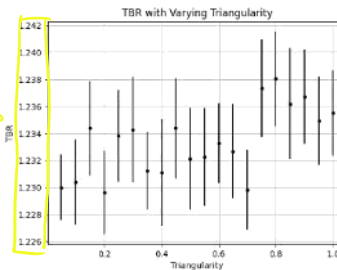
Breeder Blanket Thickness Sweep

- Increasing breeder blanket thickness unsurprisingly increased TBR
 - Increases chance of breeding reactions for outgoing neutrons
- This increased and plateaued more rapidly
 - Possibly due to thickness > mean free path of neutrons?
 - Or just that all the lower energy neutrons are expended?
- Plateaued at around 50-70cm
 - Which does match with several tokamak designs (thickness ~ 50cm)



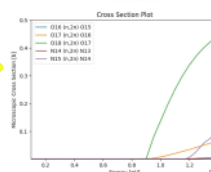
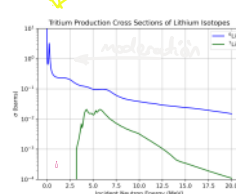
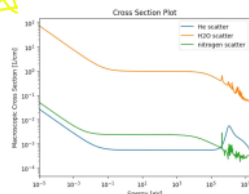
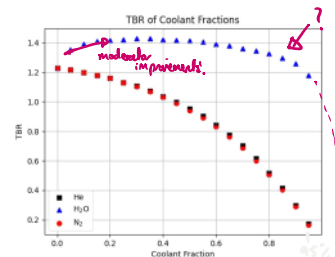
Triangularity Sweep

- Sweeping Triangularity seems to not affect TBR at all
 - The y axis is so scaled down that the error bars are visible
- This makes sense as this is a neutronics simulation, triangularity mainly affects plasma properties
- We also have used a ring source
- If we move to a plasma body source, we could get more interesting results



Coolant Proportion Thickness Sweep

- For common designs, coolant fraction by volume is ~ 30%
- We observe decreasing TBR as coolant fraction increases (so breeder material decreases)
 - Unsurprising as the no. breeding atoms available decreases
- However: the TBR was highly constant against water coolant fraction???
 - Highly uncertain about this physics of this
 - We considered the potential for the rare Oxygen multiplying reactions, however these cross sections are low and after testing enrichment we excluded this
 - We then considered the scatter cross section, and the bare cross section + the far higher density of water (liquid) meant that it is a far better scatterer.
 - This could mean that water is causing the neutrons to be moderated and scattered, and then eventually cause a very high chance of breeding reaction? Almost no neutrons escape?



- This is supported by the idea that all the neutrons are attenuated
 - Action: Build a mechanism to count how many neutrons are transmitted to confirm this
- If so, this means we have effectively created a lithium fission reactor blanket (moderated).
- The advantage of this is that we only need 20% of the normal required material to achieve the same TBR
- This means that the blanket can be ~ 3-5 times thicker,
 - Action: investigate more heterogenous models: Build a layered water and breeder material blanket structure (as this structure makes sense for cooling).

Supervisor Questions

- ☐ Discussion about heterogenous and homogenous reactor effects
- ☐ Discussion about moderated lithium fission breeder blanket design
 - ☐ Layered approach
- **Action: Boundary Conditions: Considering Reducing Revolving Angle**
 - Vacuum boundary: escapes
 - Reflective boundary: reflects
 - White boundary: reflects specularly (random direction)
 - ☒ Periodic boundary: re-enters at the opposite end
 - ☒ Upgraded to use this
 - Albedo: Vacuum & White Mix
- **Potential avenue to improve:** (could be developed using openmc_plasma_source), could develop to include a full plasma source of neutrons as opposed to single ring.
 - Also need to investigate neutron production density and maybe integrate this in
- Plasma physics and fusion energy: Jeffrey Friedburg

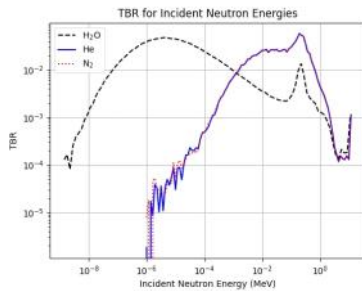
Actions List

- **Action: Develop Layered Build Granular Blanket Model**
 - **Action:** investigate more heterogenous models: Build a layered water and breeder material blanket structure (as this structure makes sense for cooling).
 - ☒ **Develop Tallying for Each layer / Reaction Rates:** Ideally have heatmap plots of reaction rates per layer!
 - ☒ Could also look at binning the reaction rates per energy of neutron, to see if all these reactions are happening following moderation.
- ☒ **Action: Boundary Conditions: Considering Reducing Revolving Angle**
- **Potential avenue to improve:** (could be developed using openmc_plasma_source), could develop to include a full plasma source of neutrons as opposed to single ring.
 - Also need to investigate neutron production density and maybe integrate this in
 - Plasma physics and fusion energy: Jeffrey Friedburg

Report Stuff

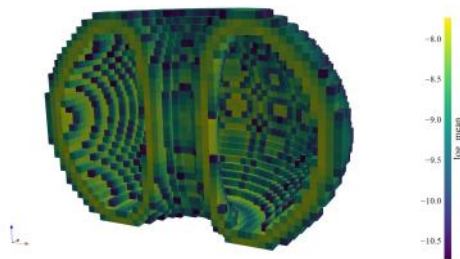
- ☐ Consider changing colours to be printer / colour blind friendly

Rhodri Completed a Study on this:



1

Completed a first attempt at this: Can see the TB decay as you go through the blanket, but would be good to try a denser mesh



Link to repository: <https://github.com/VadanKhan/fusion-neutronics-modelling/tree/main>

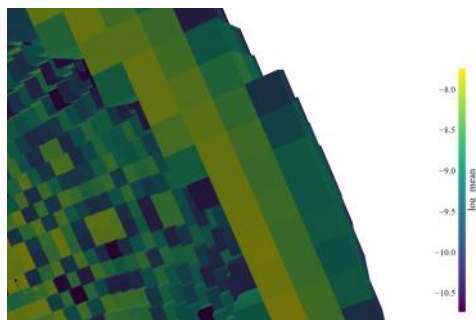
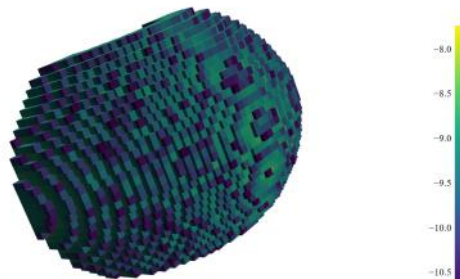
- Commit eaec12
- eaec123e32b065fd8c9bade544ccaf724daab41
- File under "materialsinvestigations.ipynb"

Link to Drive of Shared Documents:

<https://drive.google.com/drive/u/0/folders/19LqvtoSB7r2HaBN0A3UFInr3q7Z7n65p>

Relative Path of Thesis:

- Root Masters Directory (personal onedrive) \ Thesis



Labbook Week 8

18 November 2025 09:02

Progress Checkpointing

For Semester 1:

- Detailed Material Sweep (TBR & Heating)
 - o Liquid and Solid
- Coolant + Proportions
 - o Energy Binned Plot
 - ☒ o Neutron TBR Attenuation Plot: Vadan
- Geometry
 - o Blanket Thickness
 - o First wall Thickness
 - o Triangularity
- Li6Li7 Enrichment Graph
- ☒ - LiPb Neutron Multiplier: Rhodri

Data can be found in drive, under "key figures":

<https://drive.google.com/drive/u/0/folders/19Lqvt0SB7r2HaBN0A3UFInR3q7Z7n65p>

Maybe Leave for Semester 2:

- Heterogeneous Blanket
- Plasma Source

Heating Discussion --> Bromine Anomaly

- We have been able to run sweeps of heating results for each material we simulated on the base model
 - o The heating is in eV, and is the average heating per neutron (in total across the model)
 - o Not 100% sure why the heating is so much lower than 14MeV
- We see that the heating is ~ proportional to the TBR rate
 - o This makes sense as the TBR reactions (almost all in Li6) is exothermic
- However, we did see that bromine had anomalously high heating values
 - o This could be due to the higher absorption cross section
 - o Where, once absorbed, the KE of the neutron is transferred into a gamma ray and then heating
- This did however highlight that lower mass number halogens are preferable, as they absorb less neutrons
 - o $F > Cl >>> Br$

Summary ENDF-6 a js MF3-Plot Plot BR-79(N,G)BR-80,SIG MT102 QH=7.89239e+6 QI=7.89239e+6 LR=0

Validation:

Checked cross-sections in ENDF/B-VIII.0.

- Bromine has significant resonance peaks in the keV range compared to Chlorine.
- Moderation potential: Bromine is heavier, so less energy lost per collision (\$\alpha\$ parameter), but the capture gamma release is higher.
- **Decision: Result is real, not a bug. Bromine is valid for heating but poor for breeding.**

Supervisor Meeting

- Good amount of results
- Ensure the discussion links together these results to answer the physics question
- Note, the more complex model expected results is likely going to come up in the interview
- Appendix: of interest but not necessary for the main body of the report

90 ° Revolution Upgrade

In order to attempt a higher resolution mesh for the TBR heatmap, it would help to reduce the size of the reactor we simulate to reduce runtime.

We can also upgrade the boundary conditions to improve accuracy.

- Adjusted tokamak function to take angle as input
- Adjusted mesh builder to build two periodic surfaces
 - o This is better practises in nuclear simulation standards
 - o Can just change boundary arg to 'periodic', but then need to connect boundaries.
 - o (hardcoded)
- Adjusted settings builder to take rotation angle to dynamically adjust angle of plasma source
 - o Was just a full ring before!

Difference between the two mesh sizes:

1. The "Shape" Mesh (Converter Code)

Found in: converter.export_dagmc_h5m_file(...) Type: Unstructured Surface Mesh (Triangles)

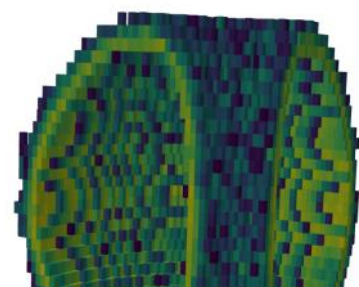
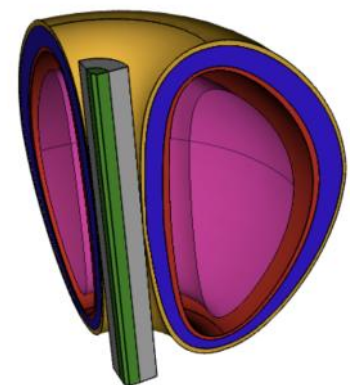
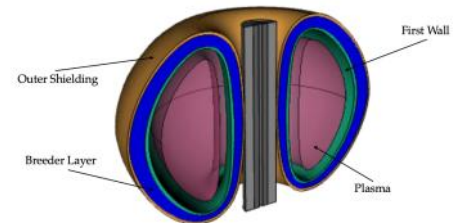
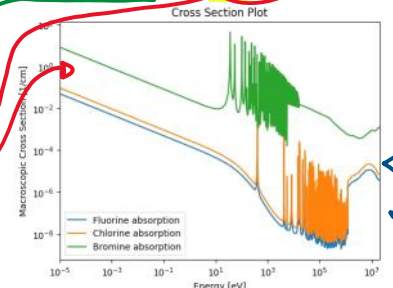
- **Purpose:** This mesh defines reality for the simulation.
- **How it works:** Your CAD model is made of perfect mathematical curves. OpenMC cannot calculate ray-tracing on perfect curves easily. So, the CadToDagmc converter turns your smooth CAD surfaces into thousands of tiny flat triangles (like a video game model).
- **The Parameters:**
 - o max_mesh_size=20.0: This tells the converter, "Don't let any triangle edge be longer than 20cm."
 - o min_mesh_size=2.0: "Don't make triangles smaller than 2cm" (prevents file bloat).
- **Impact:**
 - o **High Res (Small numbers):** Your reactor looks very smooth and circular. Rays hit the "walls" exactly where they should.
 - o **Low Res (Big numbers):** Your circular reactor might look like a hexagon or a stop sign. Rays might "leak" through gaps or bounce off at wrong angles.

2. The "Scoreboard" Mesh (Settings Code)

Found in: openmc.RegularMesh(...) Type: Structured Volume Mesh (Voxels / Cubes)

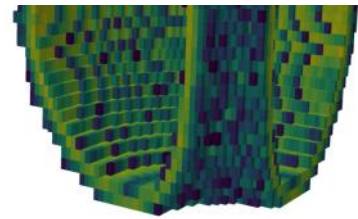
- **Purpose:** This mesh collects data. It is an imaginary grid of 3D boxes floating over your reactor.
- **How it works:** It doesn't change the geometry at all. It just sits there. When a neutron interacts with an atom, OpenMC checks, "Which box am I in?" and adds +1 to that box's score.
- **The Parameters:**
 - o dimension=[100, 100, 100]: "Divide this space into 1 million boxes."
- **Impact:**
 - o **High Res (High dimension):** You get a beautiful, high-definition heatmap picture (like a 4K photo).

	Variance	Heating	Heating Error
Li	1.368042	0.004613	1.017118e+07
Li17Pb83	1.428198	0.004642	9.286333e+06
LiF_BeF2	1.325755	0.005063	9.758002e+06
LiF_BeF2	1.329477	0.004145	9.774847e+06
LiF_PbF2	1.348896	0.003564	9.387146e+06
Li3N	1.348820	0.004821	1.001875e+07
LiF	1.361652	0.004582	9.989198e+06
LiF_BeF2_NaF	1.292672	0.003412	9.545496e+06
LiF_NaF_ZrF4	1.286461	0.003721	9.298676e+06
LiF_NaF_KF	1.251768	0.003267	9.298805e+06
LiCl_NaCl	1.219489	0.003771	9.103399e+06
LiCl_KCl	1.194766	0.003656	8.972245e+06
LiCl_KCl	1.148608	0.002515	8.731158e+06
LiCl_BeCl2	1.135484	0.003074	8.713517e+06
LiF_LiI2	1.108177	0.002771	8.266131e+06
LiCl_PbCl2	1.070092	0.002444	8.093432e+06
LiF_LiBr_NaF	1.024178	0.00385	1.149974e+07
LiF_LiBr_NaBr	1.019225	0.00291	1.148915e+07



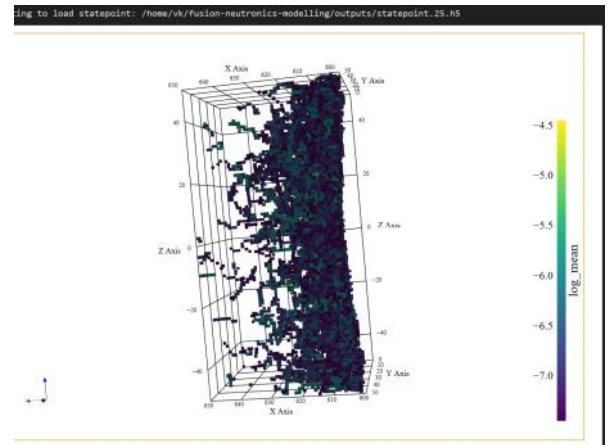
- **The Parameters:**
 - dimension=[100, 100, 100]: "Divide this space into 1 million boxes."
- **Impact:**
 - **High Res (High dimension):** You get a beautiful, high-definition heatmap picture (like a 4K photo).
 - **Low Res (Low dimension):** You get a blocky, pixelated image (like Minecraft).

Feature	Converter Mesh (DAGMC)	Settings Mesh (OpenMC Tally)
What is it?	A shell of triangles (Surface)	A block of cubes (Volume)
What does it do?	Defines the solid walls	Counts the neutrons
If you make it finer?	Physics gets more accurate (less leakage)	Pictures get sharper (higher resolution)
If you make it too fine?	Simulation runs slower (more triangles to check)	Computer runs out of RAM (too much data to store)



Heatmap Zoom Plot

- Have upgraded settings function to take pixel no. per dimension as an input arg
- Runtime seems ~ proportional to num_pixels_per_dimension^3
 - **NB:** default setting 100. If setting too high (eg. 400) the simulation runs, **but the heatmap crashes and will break my vs code install on wsl.**
- Instead moved to add in upgrade to zoom into an area as an optional input arg
 - Note that I added 'show grid' to know where to zoom
 - This is outer wall
 - However now the TBR intensity is low... perhaps because smaller boxes? Need more neutrons?



- Added 3 improvements:

Here are the three ways to fix the plot, in order of effectiveness

Option A: Fix the Scale (The Instant Fix)

Since your values are smaller (due to small boxes), your color bar expects "Big Box" numbers. You need to tell PyVista to "zoom in" on the color scale.

In your plotting code, you are likely using threshold_percent(0.001) or similar. This cuts off the low values.

- **Action:** Lower the threshold or remove it to see the "tail" of the attenuation.
- **Action:** Manually set the color range (clim) in the plot command.

```
# In your plot command:
clipped_and_thresholded.plot(
    show_grid=True,
    clim=[1e-6, 1e-4] # <--- PLAY WITH THESE NUMBERS
    # Set the 'min' very low to pick up the faint stragglers
    # Set the 'max' lower to make the front 'burn' brighter
)
```

Option B: Normalize by Volume (The "Scientific" Fix)

If you want the colors to represent **Reaction Density** (which stays the same regardless of box size), you should divide the result by the volume of the voxel.

Python

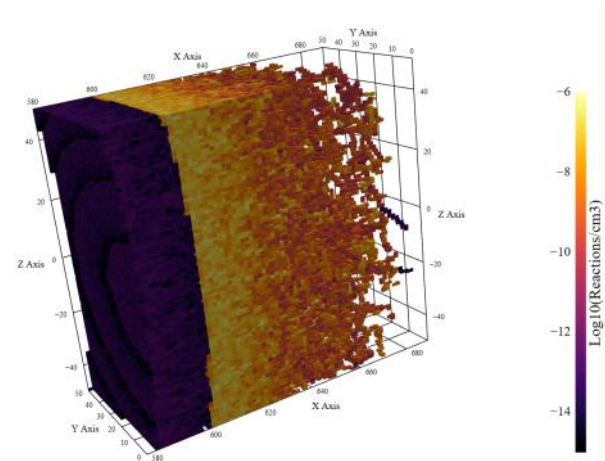
```
# Calculate volume of one voxel (in cm3)
# Total width / number of pixels
x_width = (mesh_bbox[1][0] - mesh_bbox[0][0])
voxel_side = x_width / num_pixels_per_dimension
voxel_vol = voxel_side ** 3
# Divide the mean by volume
density_values = my_tbr_tally.mean() / voxel_vol
# Now plot 'density_values' instead of 'mean'
```

This makes your plot independent of the mesh resolution.

Option C: Increase Particles (To fix "Graininess")

If your plot looks "speckled" (lots of random zero-value boxes mixed with colored ones), then you need more neutrons.

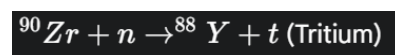
- **Current:** 5,000 particles * 25 batches = **125,000 histories**. This is quite low for a high-res mesh.
- **Recommendation:** Increase **Particles per Batch**, not Batches.
 - We used 1,000,000 neutrons for 3 batches



Purple First wall Explanation

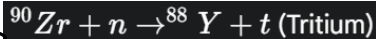
Even if the mesh were perfect, your First Wall *would* technically produce a tiny amount of Tritium.

- **The Physics:** You are bombarding the wall with 14.1 MeV neutrons. These are extremely high-energy particles.
- **The Material:** You defined your wall as **Zirconium (Zr)** (Note: You named it 'eurofer' in the code, but the composition Zr 0.98 is actually **Zircaloy**, not Eurofer steel. Eurofer is mostly Iron).
- **The Reaction:** Zirconium isotopes have a threshold reaction (n, t) at high energies. It is rare compared to Lithium, but it exists.
 - Very endothermic

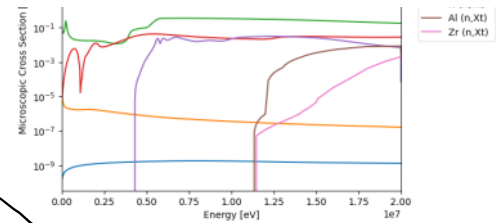


steel. Eurofer is mostly iron).

- **The Reaction:** Zirconium isotopes have a threshold reaction (n, t) at high energies. It is rare compared to Lithium, but it exists.
 - Very endothermic



- **The Magnitude:** These values will be orders of magnitude lower than the blanket, which explains why they appear as the lowest color (purple) on our heatmap.

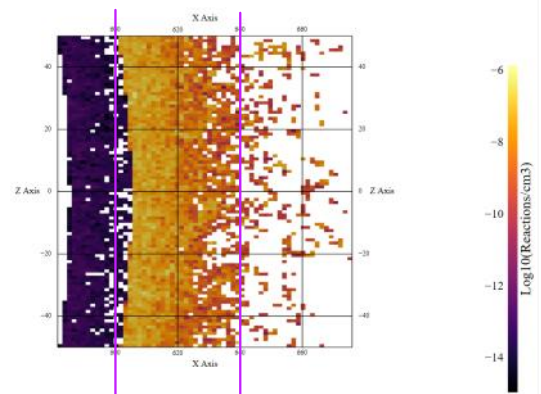


2D slicing

We then looked to make a 2D sliced plot to make a more presentable figure to clearly demonstrate the neutron attenuation effect.

You can make a plot in pyvista using mesh_plot.slice:

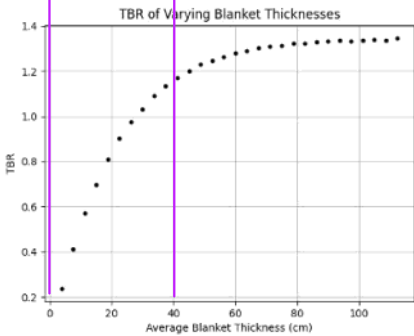
```
my_slice = mesh_plot.slice(normal='y', origin=(630, target_y, 0))
# 7. Plotting
my_slice.set_active_scalars('log(Reaction Density)')
# Optional: Threshold to remove background (e.g. values below 10^-15)
# 10^-15 => log10 is -15
valid_slice = my_slice.threshold(-15, scalars='log(Reaction Density)')
pl = pv.Plotter()
pl.add_mesh(
    valid_slice,
    cmap='inferno',
    clim=[-8, -2], # Adjust these limits based on your results (e.g. -4 to -1)
    show_edges=False,
    scalar_bar_args={'title': 'Log10(Reactions/cm3)'}
)
# Set camera to look at the slice (XZ plane)
pl.view_xz()
pl.show_grid()
pl.show()
```



However for more customisation, we can take the density values and plot them in matplotlib

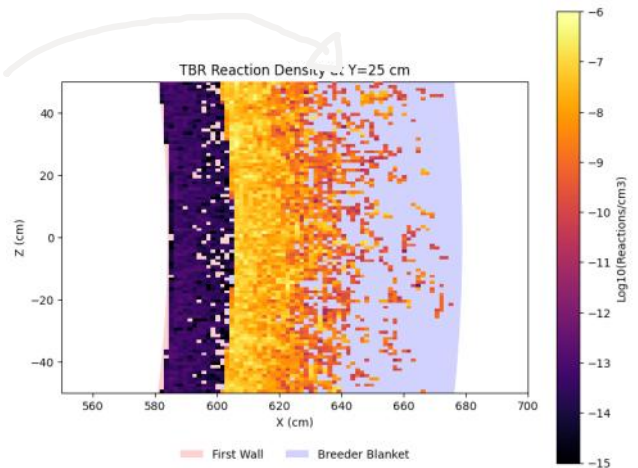
- You can use `data_3d = density_values.reshape((nz, ny, nx))` to gather the data
- You can use `plt.patches`, to create overlays to indicate the breederblanket region and first wall (manually)
- You can use `np.nan` to make the background transparent
- You can use `z` ordering to ensure that the patches are underlays

```
# Region 1: First Wall
fw_patch = patches.Wedge(
    ...
    zorder=1 # <--- KEY: Low Z-Order (Draw first)
)
# Region 2: Breeder Blanket
bb_radius = fw_radius + fw_thickness # Starts where FW ends
bb_thickness = 75
bb_patch = patches.Wedge(
    ...
    zorder=1 # <--- KEY: Low Z-Order
)
ax.add_patch(fw_patch)
ax.add_patch(bb_patch)
# Use imshow. origin='lower' is crucial so Z goes up!
im = ax.imshow(log_slice,
    origin='lower',
    extent=extent,
    cmap=my_cmap,
    vmin=-15, vmax=-6,
    zorder=10) # Adjust your color limits here
```



Overall the analysis was a great success

- Can see the neutron attenuation in the TBR attenuation halfway through the 75cm breeder blanket
- The point where the TBR reaction density mostly dies off matches with the TBR against thickness plot (both taper off after 40cm)



Link to repository: <https://github.com/VadanKhan/fusion-neutronics-modelling/tree/main>

- Commit cb2b797
- cb2b7976d46773d4dfc474d7ab0da48e33c68493
- File under "materialsinvestigations.ipynb"

Link to Drive of Shared Documents:

<https://drive.google.com/drive/u/0/folders/19Lqvt0SB7r2HaBN0A3UFINr3q7Z7n65p>

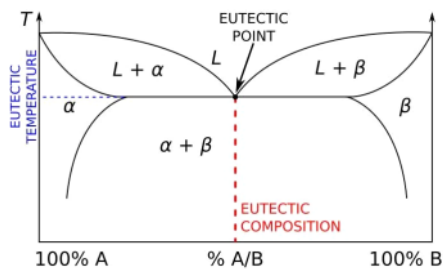
Relative Path of Thesis:

- Root Masters Directory (personal onedrive) \ Thesis

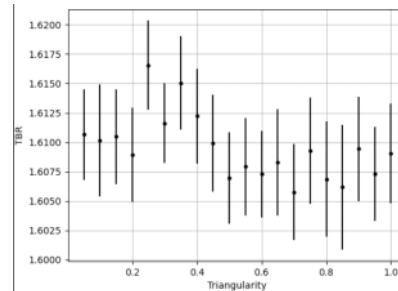
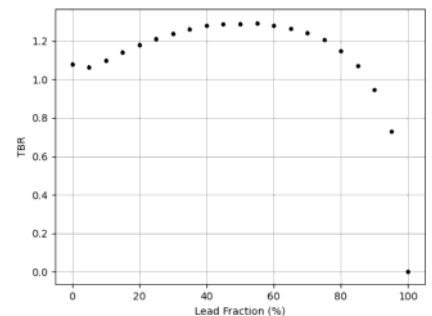
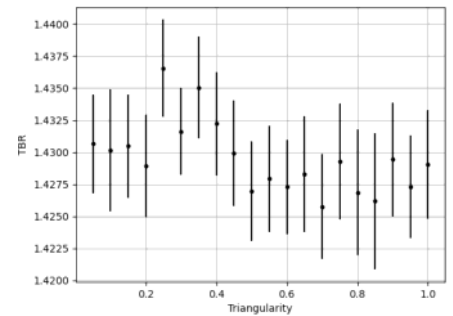
Progress Checkpointing

For Semester 1:

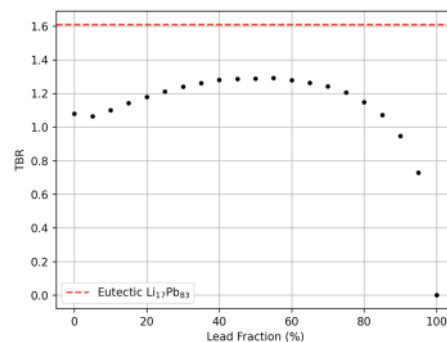
- ✓ - Detailed Material Sweep (TBR & Heating)
 - ✓ Liquid and Solid
- ✓ - Coolant + Proportions
 - ✓ Energy Binned Plot
 - ✓ Neutron TBR Attenuation Plot: Vadan
- ✓ - Geometry
 - ✓ Blanket Thickness
 - ✓ First wall Thickness
 - ✓ Triangularity: needs to be rerun with our lower first wall thickness (to increase the TBR to our other graphs to avoid questions / increase consistency)
- ✓ - Li6Li7 Enrichment Graph
- ✓ - LiPb Neutron Multiplier:
 - The LiPb Graph had a much lower general TBR
 - This is because Li17Pb83 is a "eutectic alloy", that is a special chemical property that enables the atoms to be bonded in particular way that allows higher density
 - And a lower melting point
 - When we try to sweep Pb ratios the material is no longer eutectic (only for at a certain critical point) and so the densities are lower.
 - For this reason, though the LiPb was interesting, the overall TBR is lower and so the Eutectic alloy is the sensible choice
- ✓ - Could add in the TBR for Li17Pb83 to show this



First run Triangularity Improvement
(thinner walls, same no. neutrons)



Added comparison to Eutectic Allow TBR

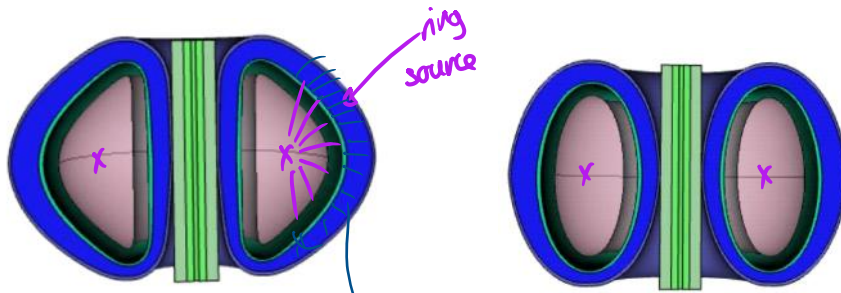


Physics Sanity Check: Why is LiPb TBR so high?

- Simulation Result: TBR ~1.61
- Manual Check: Pure weighted average density of Li + Pb ~ 9.5 g/cm³
- Eutectic Property: Li17Pb83 forms a crystal structure with higher packing efficiency.
- Database Value: density ~11.0 g/cm³ (Mas de les Valls E, Sedano LA, Batet L, Ricapito I, Aiello A, Gastaldi O, et al. Lead-lithium eutectic material database for nuclear fusion technology. Journal of Nuclear Materials. 2008 Jun 15; 376:353-7.).
- Conclusion: The simulation correctly uses the eutectic density, which explains the ~15% performance jump over the non-eutectic mixture. **Physics holds up**

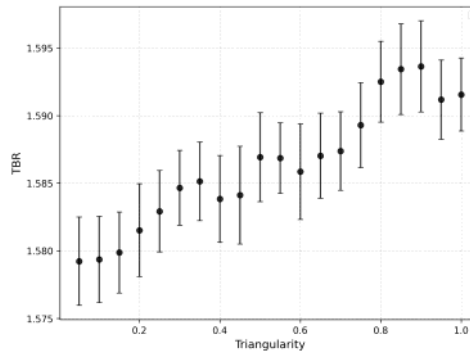
Next Semester Work

- Have a distinct Verdict for "Best TBR Setup"
- Heterogenous Blanket
- Plasma Volume Source
- More Realistic Tokamak Model
 - Testing Multiple Different Tokamak Models
- Testing Multiple Different Models to See if Transferable
- Probe and Test Leakage Fractions
- ML:
 - Reinforcement Learning Model Convergence
 - Surrogate Modelling



Second run Triangularity Improvement
Higher neutron number, updated code version, full rerun

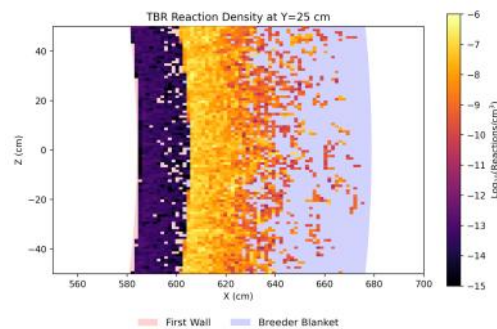
- Trend likely from the fact that the outer walls are thicker, which the ring of neutrons get more exposed to for higher triangularity
- This is not physical, as a real plasma source is a volume source, and has higher density towards the radial centre.
- This then adds evidence for the need for a plasma body source next semester!



Also reran the heatmap plots on the laptop

- Had to increase ram allocation for WSL on my laptop
- Leaving some cores (2) and some ram (4gb) safe for UI running
- However can add a healthy swap (24gb) in case of stack overflow.

Added some plotting tweaks:



Supervisor

- ☒ - Reduce point size for enrichment plot to see error bars (or cross)
- ☒ - Patches for 2D heatmap as dotted lines
- ☐ - Review literature for different lithium enrichments
- ☐ - Eutectic Alloy Melting Point
 - ☐ Look for lithium lead phase diagram and use "lever" rule to find the density

Link to repository: <https://github.com/VadanKhan/fusion-neutronics-modelling/tree/main>

- Commit dc624d7
- dc624d7b8d5d6cd50ca9fbc3e4bd0a089093e2e9
- Complete final script under materialsinvestigations.ipynb

Link to Drive of Shared Documents:

<https://drive.google.com/drive/u/0/folders/19Lqvt0SB7r2HaBN0A3UFINr3q7Z7n65p>

Relative Path of Thesis:

- Root Masters Directory (personal onedrive) \ Thesis

Labbook Week 10

03 December 2025

13:57

(week work dominated in writing up report)

First Draft Report Review (need to cut pages)

- ☒ - Introduction, do we need the signposting
- Theory too dense, move some in the appendix
 - o Mean free path?
 - ☒ o Moderation kinematics
- Referencing
 - ☒ o Remove the "available from" link
 - ☒ o Compact the link in several ways
- Too much simulation processing chain mention (dagmc not necessary)?
- ☒ - **Need to improve cross section plots**

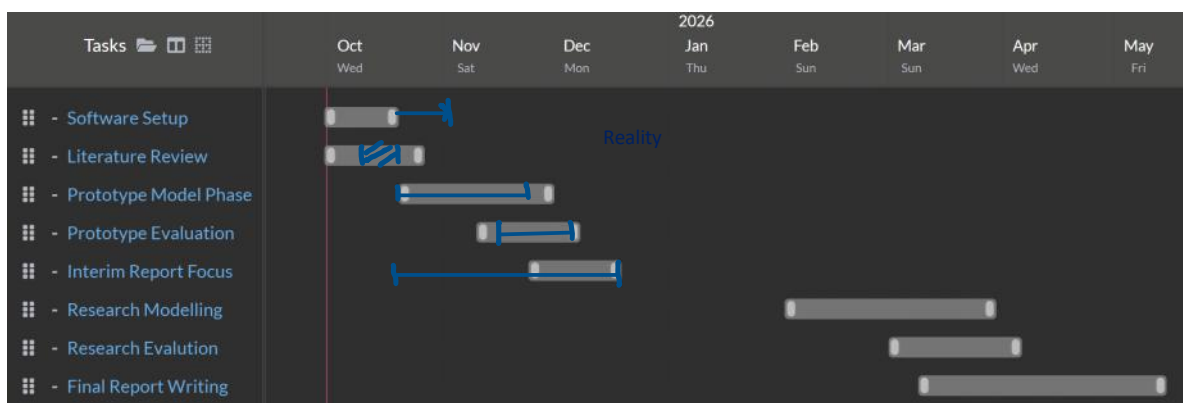
- ☐ - Need to start looking at presentation
- ☒ - Tweak / Upgrade Labbook
 - o Add some citations to labbook
 - o Link code in labbook

Supervisor meeting

- 2 columns?
- ☒ - Can force figure locations
- ☒ - Consistency with typewriter font

Project Timeline Review:

- **Planned (Week 1):** "Complete Loop" model by mid-Oct.
- **Actual:** Achieved Week 6 (delayed by DAGMC conversion issues).
- **Mitigation:** Switched to homogenous approximation to recover time on complex geometry builds.
- **Outcome:** All Semester 1 deliverables (TBR sweep, Thickness sweep) completed on schedule.



Link to repository: <https://github.com/VadanKhan/fusion-neutronics-modelling/tree/main>

- (unchanged)

Link to Drive of Shared Documents:

<https://drive.google.com/drive/u/0/folders/19LqvtoSB7r2HaBN0A3UFINr3q7Z7n65p>

Relative Path of Thesis:

- Root Masters Directory (personal onedrive) \ Thesis